

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN



Diseño y despliegue de una infraestructura de almacenamiento distribuido y observabilidad nativa de la nube mediante Infrastructure as Code (IaC)

Estudiante: Brais Fernández Reyes

Dirección: Roberto Rey Expósito

A Coruña, setembro de 2026.

A todos los que me han dado algo.

Agradecimientos

A mi familia y amigos, especialmente a Rubén, Pablo y Hugo, que me llevan acompañando toda la vida y con los que he compartido coche incontables veces para ir a la facultad. Gracias también a Lúa por conseguir darme ganas de ir a clase alguna vez.

Resumen

Este Trabajo de Fin de Grado diseña y despliega, mediante técnicas de Infrastructure as Code (IaC), una infraestructura que combina un clúster de almacenamiento de objetos distribuido (MinIO) con una pila de observabilidad nativa de la nube (Loki, Grafana, Tempo y Mimir), recolectada mediante Grafana Alloy. El entorno completo, formado por cuatro máquinas virtuales, se aprovisiona de forma automatizada y reproducible con Vagrant y Ansible, organizado en roles reutilizables, y es compatible con Debian, Rocky Linux y AlmaLinux, lo que ha permitido documentar en detalle los problemas reales de portabilidad entre distribuciones que surgen al automatizar infraestructura con estas herramientas. El sistema se valida mediante un caso de uso práctico (una aplicación de demostración que genera carga real), comprobando de forma experimental que sus métricas, logs y trazas se recolectan y correlacionan correctamente de extremo a extremo, y que el comportamiento del clúster de MinIO ante la caída de un nodo se corresponde con el esperado.

Abstract

This Bachelor's Thesis designs and deploys, using Infrastructure as Code (IaC) techniques, an infrastructure that combines a distributed object storage cluster (MinIO) with a cloud-native observability stack (Loki, Grafana, Tempo and Mimir), collected through Grafana Alloy. The complete environment, made up of four virtual machines, is automatically and reproducibly provisioned with Vagrant and Ansible, organised into reusable roles, and is compatible with Debian, Rocky Linux and AlmaLinux, which made it possible to document in detail the real portability issues that arise when automating infrastructure with these tools across distributions. The system is validated through a practical use case (a demo application that generates real load), experimentally confirming that its metrics, logs and traces are correctly

collected and correlated end to end, and that the behaviour of the MinIO cluster upon the loss of a node matches what was expected.

Palabras clave:

- Infrastructure as Code
- Almacenamiento distribuido
- Observabilidad
- MinIO
- Ansible
- Vagrant
- Docker

Keywords:

- Infrastructure as Code
- Distributed storage
- Observability
- MinIO
- Ansible
- Vagrant
- Docker

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.3	Estructura de la memoria	2
2	Estado del arte	4
2.1	Almacenamiento de objetos distribuido	4
2.2	Observabilidad nativa de la nube	5
2.2.1	Los tres pilares de la observabilidad	6
2.2.2	Alternativas existentes	6
2.3	El paradigma IaC	8
3	Tecnologías y herramientas	9
3.1	VirtualBox	9
3.2	Vagrant	10
3.3	Ansible	11
3.4	Docker	13
3.5	Docker Compose	14
3.6	MinIO	15
3.6.1	Redundancia: réplicas, RAID y codificación por borrado	15
3.7	El stack LGTM	17
3.7.1	Loki	17
3.7.2	Mimir	17
3.7.3	Tempo	18
3.7.4	Grafana	18
3.7.5	Lenguajes de consulta: PromQL, LogQL y TraceQL	19
3.7.6	Grafana Alloy	20
3.8	OpenTelemetry	20

3.9	Herramientas y lenguajes de apoyo	21
3.9.1	Git	21
3.9.2	Ruby	21
3.9.3	YAML	21
3.9.4	Bash	21
4	Metodología	22
4.1	Metodología	22
4.2	Fases principales del trabajo	23
4.3	Planificación	24
4.4	Presupuesto	26
4.5	Análisis de riesgos	27
4.6	Resumen de ejecución y desviaciones	28
5	Diseño y arquitectura	29
5.1	Requisitos	29
5.2	Arquitectura lógica	29
5.2.1	Elección de minio-1 como nodo controlador	31
5.2.2	Selección del sistema operativo como parámetro global	31
5.2.3	MinIO como servicio nativo del sistema operativo	31
5.3	Diseño del <i>pipeline</i> de observabilidad	32
5.3.1	Justificación de la exclusión de Alloy en las trazas de TNS	33
5.4	Diseño de red	34
5.5	Alta disponibilidad de MinIO	35
5.5.1	Aritmética de fragmentos y quórum para dos nodos	36
5.6	Alcance y seguridad	37
6	Implementación	38
6.1	Aprovisionamiento de la infraestructura base	38
6.1.1	Aprovisionamiento con Vagrant	38
6.1.2	Automatización con Ansible: de <i>playbooks</i> a roles	41
6.2	Almacenamiento y observabilidad	44
6.2.1	Rol <code>minio</code>	44
6.2.2	Rol <code>grafana_alloy</code>	47
6.2.3	Rol <code>lgtm_stack</code>	49
6.2.4	Integración con MinIO como <i>backend</i> S3	52
6.2.5	Adaptación de la aplicación de demostración TNS	53
6.2.6	Correlación de trazas y logs en Grafana	55

6.3	Problemas de compatibilidad	59
6.3.1	<code>firewalld</code> y las tablas persistentes de <code>nftables</code>	60
6.3.2	SDK de Python para Docker y el esquema <code>http+docker</code>	61
6.3.3	<code>secure_path</code> de <code>sudo</code> y binarios en <code>/usr/local/bin</code>	62
6.4	Herramienta de validación automatizada	63
7	Validación y pruebas	66
7.1	Validación de extremo a extremo	66
7.1.1	Latencia de escritura en el <i>backend</i> S3	67
7.2	Validación multi-sistema operativo	68
7.3	Caída de un nodo en MinIO	69
7.4	Idempotencia del despliegue	73
8	Conclusiones	77
8.1	Situación final del trabajo	77
8.2	Lecciones aprendidas	77
8.3	Conocimientos adquiridos	78
8.4	Relación con las competencias de la mención	79
8.5	Líneas futuras	79
	Lista de acrónimos	81
	Glosario	82
	Bibliografía	83

Índice de figuras

3.1	Arquitectura de Ansible: el nodo controlador ejecuta el <i>playbook</i> sobre las máquinas del inventario a través de conexiones SSH puntuales	12
3.2	Panel del clúster de MinIO en Grafana, aprovisionado automáticamente durante el despliegue	19
4.1	Diagrama de Gantt: planificación inicial frente a ejecución real	25
5.1	Arquitectura lógica del sistema: cuatro nodos agrupados en dos planos e interconectados por una red privada	30
5.2	Pipeline de telemetría: las tres señales fluyen desde <i>app-monitor-*</i> hacia los almacenes LGTM que se ejecutan en <i>minio-1</i> , con el clúster MinIO (distribuido entre <i>minio-1</i> y <i>minio-2</i>) como <i>backend</i> S3 y la ruta directa de trazas Jaeger hacia Tempo	33
7.1	Correlación de una traza de TNS con sus logs en Grafana	67
7.2	Panel <i>Node Exporter Full</i> en Grafana, con las métricas de sistema de <i>app-monitor-1</i> servidas desde Mimir	68
7.3	Panel Health Breakdown del <i>dashboard</i> de MinIO en Grafana con <i>minio-2</i> detenido: un disco en línea, igualando el quórum de lectura y por debajo del de escritura	71

Índice de tablas

2.1	Comparación de alternativas de almacenamiento de objetos autoalojado compatible con S3	5
2.2	Comparación de familias de soluciones de observabilidad	7
4.1	Planificación temporal y esfuerzo estimado del proyecto (2026)	25
4.2	Esfuerzo estimado frente a esfuerzo real, en horas, por sprint	26
4.3	Estimación de presupuesto del proyecto	27
5.1	Fragmentos de datos (k) y paridad (m), quórum y tolerancia a la caída de nodos: el clúster de dos discos de este TFG y, como referencia, uno de cuatro discos	37
6.1	Topología de las cuatro máquinas virtuales del entorno	39
6.2	Roles de Ansible y su responsabilidad	41
7.1	Matriz de validación por sistema operativo, con la evidencia que respalda cada comprobación	69

Índice de listados

3.1	Vagrantfile de ejemplo: una máquina virtual con su imagen base, su red, sus recursos y un aprovisionamiento inicial	10
3.2	Playbook de ejemplo: dos tareas sobre un grupo de hosts del inventario	12
3.3	docker-compose.yml de ejemplo: una aplicación web y su base de datos . .	14
6.1	Selección de sistema operativo en config.yml	38
6.2	Extracto del Vagrantfile: las cuatro máquinas desde MACHINES	39
6.3	Estructura de directorios del código Ansible basado en roles	42
6.4	Órdenes de despliegue completo y parcial sobre site.yml	43
6.5	Extracto de site.yml: asociación de grupos de hosts y roles	43
6.6	Extracto del rol minio: usuario de servicio, preparación del disco y configuración del modo distribuido	45
6.7	Extracto de la plantilla config.alloy.j2: bloque condicional según el inventario	47
6.8	Extracto del rol grafana_alloy: renderizado de la configuración y despliegue del contenedor	47
6.9	Extracto del rol lgtm_stack: creación de los <i>buckets</i> y despliegue del stack con Docker Compose	49
6.10	Extracto de la plantilla del docker-compose.yml del stack LGTM	50
6.11	Extracto de la configuración de almacenamiento S3 de Loki, Mimir y Tempo hacia MinIO	52
6.12	Fichero docker-compose.yml de la aplicación TNS empleado en este proyecto	54
6.13	Correlación bidireccional traza-logs en los <i>datasources</i> de Loki y Tempo (<i>datasources.yml</i>)	55
6.14	Activación del <i>metrics-generator</i> de Tempo (<i>tempo.yml</i>)	59
6.15	Neutralización correcta de <i>firewalld</i> (rol <i>firewall</i>)	60
6.16	Fijado de versión de <i>requests</i> en el rol <i>docker</i>	61
6.17	Tareas del rol <i>minio</i> que dejan ambos binarios accesibles bajo <i>secure_path</i>	63

6.18	Extracto de <code>test-provision.sh</code> : comprobaciones de ingesta y de persistencia en MinIO	64
7.1	Comprobación directa de ingesta en Mimir y Loki	66
7.2	Forzado manual del vaciado a MinIO	67
7.3	Salida de <code>mc admin info</code> con <code>minio-2</code> detenido	70
7.4	Errores de escritura hacia MinIO registrados por Loki, Mimir y Tempo con <code>minio-2</code> detenido	71
7.5	Extracto y resumen final de una segunda ejecución de <code>site.yml</code> sin cambios, sobre Rocky Linux 9.6	74

Introducción

EN este primer capítulo de la memoria se pone en contexto el presente [Trabajo de Fin de Grado \(TFG\)](#): se expone la motivación que ha llevado a su desarrollo, se enumeran los objetivos, general y específicos, que persigue, y se describe la estructura del resto del documento.

1.1 Motivación

El almacenamiento de objetos distribuido y la observabilidad de sistemas distribuidos son dos de los pilares habituales de cualquier infraestructura nativa de la nube moderna y, sin embargo, rara vez se estudian de forma conjunta: la mayoría de los materiales docentes y de la documentación técnica disponible tratan el almacenamiento de objetos ([Simple Storage Service \(S3\)](#) y sus alternativas) y la observabilidad (métricas, logs y trazas) [1] como dominios independientes, cada uno con su propio conjunto de herramientas. Este TFG parte de la idea de que ambos dominios se entienden mejor cuando se despliegan y se estudian juntos, sobre un mismo caso de uso práctico: un sistema de almacenamiento de objetos que, además, sirve como *backend* de persistencia de su propia plataforma de [observabilidad](#).

A esto se añade una motivación de índole más práctica. La automatización de infraestructura mediante el paradigma [Infrastructure as Code \(IaC\)](#) [2] se presenta con frecuencia, tanto en cursos como en la documentación de proveedores y en la literatura divulgativa, como una solución que funciona de manera uniforme sobre cualquier sistema operativo. La experiencia de portar un mismo despliegue de Ansible entre tres distribuciones Linux distintas, documentada en detalle en este trabajo, matiza esa idea: la automatización reduce el esfuerzo, los errores y los tiempos necesarios para reproducir un despliegue, pero no elimina por sí sola las diferencias reales entre plataformas, que deben identificarse y tratarse de forma explícita.

1.2 Objetivos

El objetivo general de este TFG es diseñar y desplegar, mediante **IaC**, una infraestructura de almacenamiento distribuido y observabilidad nativa de la nube, validada con un caso de uso práctico. Este objetivo general se concreta en cuatro objetivos específicos:

1. **Diseño de la arquitectura lógica:** definir la interconexión y el dimensionamiento de los nodos que componen el sistema, justificando cada decisión de diseño en función de los requisitos del proyecto.
2. **Aprovisionamiento base automatizado y reproducible**, mediante Vagrant y Ansible, sobre varios sistemas operativos.
3. **Despliegue de un clúster distribuido de MinIO junto a un stack de observabilidad **Loki, Grafana, Tempo, Mimir (LGTM)****, mediante roles de Ansible reutilizables.
4. **Validación mediante un caso de uso práctico:** desplegar una aplicación de demostración y comprobar que su telemetría (métricas, logs y trazas) se recolecta y se correlaciona correctamente de extremo a extremo.

1.3 Estructura de la memoria

La estructura del presente documento se define de la siguiente manera:

1. **Introducción:** presenta el contexto del **TFG**, la motivación que ha llevado a su desarrollo, los objetivos que persigue y la estructura de la propia memoria.
2. **Estado del arte:** sitúa el trabajo en el contexto de las alternativas existentes en el almacenamiento de objetos distribuido, la observabilidad nativa de la nube y la automatización de infraestructura, y justifica las opciones finalmente elegidas.
3. **Tecnologías y herramientas:** describe esas tecnologías, desde VirtualBox, Vagrant, Ansible, Docker y Docker Compose hasta MinIO, el stack LGTM con Grafana Alloy y las herramientas y lenguajes de apoyo (Git, Ruby, YAML y Bash), explicando qué son, cómo funcionan y cómo se usan concretamente en este trabajo.
4. **Metodología y planificación del trabajo:** describe la metodología seguida, la planificación temporal frente a la ejecución real, el presupuesto y el análisis de riesgos.
5. **Diseño y arquitectura:** presenta la arquitectura lógica del sistema y justifica las principales decisiones de diseño.

6. **Implementación:** detalla cómo se ha materializado dicha arquitectura mediante Vagrant y Ansible, incluyendo los problemas de compatibilidad entre sistemas operativos encontrados durante el desarrollo.
7. **Validación y pruebas:** recoge las pruebas realizadas para verificar el correcto funcionamiento del sistema desplegado.
8. **Conclusiones:** resume la situación final del trabajo, las lecciones aprendidas y las líneas futuras.

Estado del arte

ESTE capítulo sitúa el trabajo en el contexto de tres áreas tecnológicas: el almacenamiento de objetos distribuido, la observabilidad nativa de la nube, y la automatización de infraestructura mediante el paradigma IaC. En cada una se revisan las alternativas existentes y se justifica la elección realizada en este TFG, cuyo funcionamiento interno se detallará después en el capítulo 3.

2.1 Almacenamiento de objetos distribuido

El almacenamiento de objetos, frente al almacenamiento en bloque o en fichero tradicional, organiza los datos como unidades independientes (objetos) accesibles mediante una API HTTP, habitualmente compatible con la interfaz de S3, el servicio de almacenamiento de objetos en la nube de Amazon Web Services [3]. Esta compatibilidad de API se ha convertido, de facto, en un estándar de la industria: la mayoría de las herramientas que necesitan un *backend* de almacenamiento de objetos, entre ellas los propios componentes del stack LGTM, asumen una interfaz compatible con S3, independientemente del proveedor real que la sirva.

Entre las alternativas de almacenamiento de objetos autoalojado (*self-hosted*) compatibles con S3 destacan:

- **MinIO** [4], la opción empleada en este TFG: un servidor de código abierto, escrito en Go y distribuido como un único binario, pensado específicamente para desplegarse en modo distribuido con codificación por borrado (*erasure coding*). Su edición Community/*bare-metal*, distribuida directamente desde su repositorio de GitHub, no requiere licencia comercial. Su edición comercial, por su parte, pasó a denominarse **AIstor** [5], orientada de forma explícita a cargas de trabajo de inteligencia artificial y con soporte comercial para empresas.
- **Ceph** [6], un sistema de almacenamiento distribuido de propósito más general (sopor-

ta también almacenamiento en bloque y en fichero), cuya arquitectura es considerablemente más compleja de operar, pues requiere gestionar por separado componentes como monitores, gestores y varios tipos de demonios de almacenamiento, y está pensada para despliegues de mayor escala que el de este TFG.

- Alternativas más recientes y de menor huella, como **SeaweedFS** o **Garage**, orientadas a escenarios con menos recursos disponibles que los clústeres tradicionales de Ceph.

La tabla 2.1 resume las diferencias principales entre estas alternativas en los aspectos más relevantes para el proyecto.

Criterio	MinIO	Ceph	SeaweedFS	Garage
Lenguaje	Go	C++	Go	Rust
Licencia	AGPLv3	LGPL 2.1	Apache 2.0	AGPLv3
Tipos de almacenamiento	Objetos (S3)	Objetos, bloque y fichero	Objetos y ficheros	Objetos (S3)
Redundancia	<i>Erasure coding</i>	Replicación y <i>erasure coding</i>	Replicación (EC opcional)	Replicación
Complejidad operativa	Baja (binario único)	Alta (varios demonios)	Media	Baja
Huella de recursos	Media	Alta	Baja	Muy baja
Orientación típica	<i>Backend</i> S3 nativo de la nube	Almacenamiento unificado a gran escala	Gran volumen de ficheros pequeños	Autoalojado geo-distribuido, pocos recursos

Tabla 2.1: Comparación de alternativas de almacenamiento de objetos autoalojado compatible con S3

La elección de MinIO para este TFG responde tanto a su simplicidad de despliegue como a su papel, cada vez más habitual, como *backend* de almacenamiento por defecto de herramientas de observabilidad de código abierto, lo que permite en este proyecto validar a la vez el propio MinIO y su integración real con dichas herramientas.

2.2 Observabilidad nativa de la nube

La **observabilidad** es la capacidad de conocer el estado interno de un sistema a partir de los datos que este emite hacia el exterior. Antes de revisar las soluciones disponibles conviene precisar los tres tipos de señal sobre los que se apoyan todas ellas, ya que las diferencias entre familias de herramientas se explican en buena medida por cómo tratan cada una de esas señales.

2.2.1 Los tres pilares de la observabilidad

La observabilidad de sistemas distribuidos se apoya habitualmente en tres tipos de señal, con frecuencia descritos como sus «tres pilares» de la [observabilidad](#) [1]:

- Una **métrica** es un valor numérico medido periódicamente y asociado a un instante de tiempo: el uso de CPU de una máquina, el número de peticiones por segundo que atiende un servicio. La secuencia de valores de una misma métrica a lo largo del tiempo forma una serie temporal, identificada por un nombre y un conjunto de etiquetas (*labels*) que la cualifican (de qué host procede, a qué servicio pertenece). Las métricas son compactas y agregables: resultan idóneas para detectar que algo va mal (una tasa de errores que sube, una latencia que crece) y para representar tendencias en paneles y alertas, pero no explican el detalle de ningún caso individual. Este modelo de datos es el de **Prometheus** [7], el sistema de monitorización de referencia en entornos nativos de la nube, cuyo ecosistema (formato de exposición, protocolos, lenguaje de consulta) adoptan buena parte de las soluciones actuales.
- Un **log** es el registro textual de un evento concreto, emitido por una aplicación o por el sistema operativo junto con su marca de tiempo: un error con su pila de ejecución, una petición HTTP atendida, un intento de inicio de sesión. Los logs aportan el máximo detalle sobre qué ha ocurrido exactamente en un caso concreto, a cambio de un volumen de datos mucho mayor y de un coste de consulta más alto que el de las métricas.
- Una **traza** (*trace*) reconstruye el recorrido completo de una petición individual a través de los distintos servicios que participan en atenderla. Se compone de *spans*: intervalos con inicio, duración y servicio responsable, anidados unos dentro de otros (la llamada a la base de datos dentro de la petición web que la provocó), y todos los *spans* de una misma petición comparten un identificador común, el `traceID`. Las trazas responden a dónde, dentro de una cadena de llamadas entre servicios, se ha producido un fallo o un cuello de botella.

La correlación entre estas tres señales (poder pasar, por ejemplo, de una traza concreta a los logs generados durante esa misma petición) es lo que distingue a una plataforma de observabilidad moderna de la simple monitorización de métricas aislada.

2.2.2 Alternativas existentes

En este espacio conviven varias familias de soluciones:

- Plataformas SaaS comerciales, como **Datadog** o **New Relic**, que ofrecen una experiencia integrada a cambio de una dependencia de proveedor y un coste recurrente ligado al volumen de datos.

- El stack **ELK/EFK** (Elasticsearch, Logstash/Fluentd, Kibana), tradicionalmente centrado en logs mediante indexación de texto completo, al que se han ido añadiendo capacidades de métricas y trazas.
- El stack **LGTM** [8] (Loki, Grafana, Tempo, Mimir) de Grafana Labs, empleado en este TFG: cada componente está especializado en un único tipo de señal, unificados en una única interfaz de consulta y visualización (Grafana). A diferencia de Elasticsearch, Loki no indexa el contenido completo de los logs, sino únicamente sus etiquetas, lo que reduce considerablemente su coste operativo a cambio de consultas de texto completo más limitadas, tal y como se explicará con más detalle en la sección 3.7.

La tabla 2.2 contrasta estas tres familias de soluciones en los criterios que motivan la elección de LGTM.

Criterio	LGTM	ELK/EFK	Datadog / New Relic
Modelo de despliegue	Autoalojado	Autoalojado	SaaS gestionado (nube del proveedor)
Coste	Sin licencia (solo infraestructura propia)	Sin licencia en su edición básica (solo infraestructura propia)	Suscripción recurrente por volumen de datos
Señales cubiertas	Métricas, logs y trazas (diseño nativo)	Logs; métricas y trazas incorporadas después	Métricas, logs y trazas (integradas)
Indexación de logs	Solo etiquetas (índice ligero)	Texto completo (índice invertido)	Gestionada por el proveedor
Almacenamiento	Backend de objetos S3 (p. ej. MinIO)	Gestionado por Elasticsearch (disco)	Gestionado por el proveedor
Coste operativo en recursos	Bajo	Alto (indexación de texto completo)	Externalizado al proveedor
Dependencia de proveedor	Baja	Baja	Alta (<i>vendor lock-in</i>)

Tabla 2.2: Comparación de familias de soluciones de observabilidad

La recolección de estas tres señales se realiza en este proyecto mediante Grafana Alloy [9], un agente unificado compatible de forma nativa con OpenTelemetry [10], que se describen en detalle en las secciones 3.7.6 y 3.8, respectivamente.

2.3 El paradigma IaC

El paradigma **IaC** es la práctica de definir y gestionar infraestructura (recursos de cómputo, redes, configuración de software) mediante ficheros de definición versionables, en lugar de mediante configuración manual [2]. Dentro de la **IaC** conviene distinguir dos capas habitualmente complementarias:

- Herramientas de **aprovisionamiento de recursos**, como **Vagrant** [11] (empleado en este TFG para gestionar máquinas virtuales locales sobre VirtualBox) o **Terraform/OpenTofu** (orientadas principalmente al aprovisionamiento declarativo de recursos de todo tipo, no solo máquinas, sino también contenedores, redes, balanceadores de carga o bases de datos gestionadas, en proveedores de nube pública, mediante un modelo de estado remoto poco adecuado para un entorno de laboratorio completamente local como el de este proyecto).
- Herramientas de **gestión de la configuración**, responsables de instalar y configurar software dentro de las máquinas ya existentes. Aquí destaca **Ansible** [12], empleado en este TFG, frente a alternativas como **Puppet** o **Chef**. La diferencia principal es arquitectónica: Ansible es *agentless* y opera mediante conexiones SSH puntuales desde un nodo controlador, mientras que Puppet y Chef requieren la instalación permanente de un agente en cada nodo gestionado. Para un entorno de laboratorio como el de este TFG, en el que las máquinas se crean y destruyen con frecuencia, el modelo *agentless* evita tener que gestionar el ciclo de vida de dicho agente. La arquitectura de Ansible se detalla en la sección 3.3.

Dentro de Ansible, la organización del código en roles, frente a *playbooks* planos con una única lista de tareas, es la práctica recomendada por la propia documentación oficial del proyecto para cualquier base de código que supere un tamaño trivial, precisamente por los problemas de mantenibilidad y duplicación que se documentan de forma práctica en el capítulo 6 de esta memoria.

Tecnologías y herramientas

EL capítulo 2 ha situado las tecnologías empleadas en este TFG frente a sus alternativas y ha justificado por qué se eligieron. Este capítulo profundiza ya en ellas: describe, para cada una, qué es, cómo funciona a un nivel suficiente para entender el resto de la memoria, y cómo se usa concretamente en este trabajo.

3.1 VirtualBox

Un hipervisor es la capa de software responsable de crear y ejecutar máquinas virtuales: reparte los recursos físicos de una máquina real (CPU, memoria, disco, red) entre varias máquinas virtuales, cada una de las cuales ejecuta su propio sistema operativo de forma aislada, como si dispusiera de hardware dedicado. Tradicionalmente se distinguen dos tipos:

- Los hipervisores de **tipo 1** (o *bare-metal*) se ejecutan directamente sobre el hardware físico, sin un sistema operativo anfitrión por debajo. Es el caso de VMware ESXi, Microsoft Hyper-V o KVM. Son los habituales en entornos de producción y centros de datos, ya que ofrecen mejor rendimiento y aislamiento.
- Los hipervisores de **tipo 2** (o *hosted*) se ejecutan como una aplicación más sobre un sistema operativo anfitrión ya instalado. Introducen una capa adicional, puesto que las peticiones de las máquinas virtuales pasan por el sistema operativo anfitrión, a cambio de una instalación y un manejo mucho más sencillos, lo que los hace idóneos para entornos de escritorio, desarrollo y laboratorio.

VirtualBox [13] es un hipervisor de tipo 2, de código abierto y multiplataforma, que permite crear y ejecutar máquinas virtuales completas sobre el equipo anfitrión, cada una con su propio disco virtual, su propia interfaz de red virtual y sus propios recursos de CPU y memoria. En este proyecto, VirtualBox es el motor de virtualización real sobre el que se ejecutan las cuatro máquinas del entorno, un uso de laboratorio local que encaja perfectamente en el

escenario para el que los hipervisores de tipo 2 resultan idóneos. En el día a día, no obstante, no se interactúa con él de forma directa, sino a través de Vagrant, que actúa como una capa de abstracción por encima.

El recurso a la virtualización responde a una necesidad puramente material: la arquitectura que se diseña en el capítulo 5 está formada por cuatro servidores independientes, y no se dispone de cuatro máquinas físicas para desplegarla. Cada máquina virtual sustituye, por tanto, a un servidor físico, con su propio sistema operativo, su propio disco y su propia dirección de red. Nada de lo que se automatiza después con Ansible depende de que esos servidores sean virtuales: los mismos roles, ejecutados contra un inventario de máquinas físicas alcanzables por SSH, producirían el mismo resultado. La virtualización solo afecta a la fase de creación de las máquinas, que es precisamente la que gestiona Vagrant.

3.2 Vagrant

Vagrant [11] es una herramienta IaC de línea de comandos (CLI) que gestiona el ciclo de vida completo de máquinas virtuales (VMs) ejecutadas en un entorno local, desde su creación y arranque hasta su destrucción, pasando por el aprovisionamiento inicial, todo ello mediante un único fichero de definición escrito en Ruby: el `Vagrantfile`. A diferencia de herramientas como Terraform, que emplean un lenguaje declarativo propio, el `Vagrantfile` no es una plantilla declarativa sino un programa Ruby que Vagrant evalúa al arrancar, por lo que admite construcciones procedimentales (bucles, condicionales o la lectura de ficheros externos). Vagrant no virtualiza nada por sí mismo, sino que delega esa tarea en un proveedor de virtualización subyacente (*provider*), VirtualBox en este caso, y añade por encima una capa de automatización común a cualquier proveedor: definición de la red, de los discos, de las carpetas compartidas entre el anfitrión y la máquina virtual, y de los pasos de aprovisionamiento inicial que deben ejecutarse la primera vez que la máquina arranca.

El listado 3.1 recoge un `Vagrantfile` mínimo, ajeno a este proyecto, que ilustra esa estructura sobre una única máquina virtual.

```
1 Vagrant.configure("2") do |config|
2   # Imagen base ("box") de la que parte la maquina virtual
3   config.vm.box = "debian/bookworm64"
4
5   config.vm.define "web" do |web|
6     web.vm.hostname = "web"
7     web.vm.network "private_network", ip: "192.168.56.20"
8
9     # Recursos que el proveedor asigna a la maquina
10    web.vm.provider "virtualbox" do |vb|
11      vb.memory = 1024
```

```
12     vb.cpus = 1
13     end
14
15     # Aprovisionamiento inicial: se ejecuta al crear la maquina
16     web.vm.provision "shell", inline: "apt-get update && apt-get
      install -y nginx"
17     end
18 end
```

Listado 3.1: Vagrantfile de ejemplo: una máquina virtual con su imagen base, su red, sus recursos y un aprovisionamiento inicial

Cada bloque anidado se corresponde con uno de los aspectos que Vagrant abstrae: la *box* de la que se parte, la red, los recursos que se piden al proveedor y los pasos de aprovisionamiento. A partir de esa definición, la orden `vagrant up` crea la máquina, la arranca y ejecuta el *provisioner*; `vagrant destroy` la elimina por completo. Como el fichero se versiona junto al resto del código del proyecto, cualquier equipo con Vagrant y el proveedor correspondiente reproduce el mismo entorno a partir de él.

El Vagrantfile desarrollado en este proyecto define las cuatro máquinas virtuales del entorno, cuya arquitectura y dimensionamiento se presentan en el capítulo 5. Las cuatro se generan a partir de una única estructura de datos Ruby, evitando así repetir la misma definición cuatro veces con pequeñas variaciones. Cada máquina recibe una dirección IP estática dentro de la red privada del proyecto, su propia asignación de memoria y CPU, un disco virtual adicional en el caso de los nodos de MinIO, y una serie de *provisioners* de tipo `shell` que instalan los paquetes base del sistema operativo. El propio sistema operativo de las cuatro máquinas se selecciona en tiempo de lectura del Vagrantfile a partir de un fichero externo de configuración, lo que permite reutilizar exactamente el mismo Vagrantfile para Debian, Rocky Linux y AlmaLinux.

3.3 Ansible

Ansible [12] es una herramienta IaC de gestión de la configuración: una vez las máquinas ya existen (en este TFG, creadas por Vagrant), Ansible se encarga de instalar y configurar el software dentro de ellas. Su característica arquitectónica más relevante es que es *agentless*: no requiere instalar ni mantener ningún proceso permanente en las máquinas gestionadas. En su lugar, un único nodo controlador se conecta por SSH a cada máquina de destino en el momento de ejecutar un *playbook*, transfiere un pequeño módulo Python autocontenido (empaquetado mediante una técnica llamada `Ansible1Z`) y lo ejecuta de forma remota. Terminada la ejecución, no queda ningún proceso de Ansible corriendo en la máquina gestionada.

Los dos ficheros que definen cualquier ejecución de Ansible son el inventario, que enumera las máquinas gestionadas y las agrupa por función, y el *playbook*, que describe qué debe hacerse sobre cada uno de esos grupos. La figura 3.1 resume esta arquitectura.

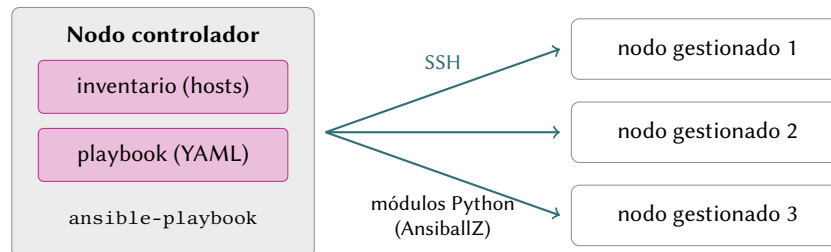


Figura 3.1: Arquitectura de Ansible: el nodo controlador ejecuta el *playbook* sobre las máquinas del inventario a través de conexiones SSH puntuales

Cada unidad de trabajo (una tarea) invoca un módulo: una pieza de código que encapsula una operación concreta (por ejemplo, `docker_container` para gestionar un contenedor Docker, o `systemd` para gestionar un servicio) y que, en teoría, es idempotente: declara el estado final deseado, no los pasos para alcanzarlo, y es responsabilidad del propio módulo comparar ese estado deseado con el estado real del sistema y actuar solo si difieren. Como se discute en la sección 7.4 de esta memoria, esta garantía depende de cómo se emplee cada módulo, y no es automática por el mero hecho de usar Ansible.

Los *playbooks* se escriben en YAML, el formato declarativo de serialización de datos descrito en la sección 3.9.3. El listado 3.2 muestra un *playbook* de ejemplo, con dos tareas aplicadas a un grupo de máquinas del inventario.

```

1 - name: Configuración básica de los servidores web
2   hosts: webserver      # grupo definido en el inventario
3   become: true          # ejecutar las tareas como root
4
5   tasks:
6     - name: Instalar el servidor web
7       ansible.builtin.package:
8         name: nginx
9         state: present    # estado final deseado, no pasos
10
11    - name: Arrancar el servicio y habilitarlo en el arranque
12      ansible.builtin.systemd:
13        name: nginx
14        state: started
15        enabled: true

```

Listado 3.2: *Playbook* de ejemplo: dos tareas sobre un grupo de hosts del inventario

Ninguna de las dos tareas indica cómo alcanzar el estado descrito: se limitan a declararlo (el paquete presente, el servicio arrancado y habilitado) y es cada módulo el que comprueba el estado real de la máquina y actúa solo si difiere. Ejecutar dos veces seguidas este *playbook* no debería, por tanto, producir ningún cambio en la segunda ejecución.

El bloque completo del listado anterior, es decir, la asociación entre un grupo de máquinas del inventario y la lista de tareas que deben aplicarse sobre él, recibe en la terminología de Ansible el nombre de *play* (literalmente, «jugada»). Un *playbook* es exactamente eso: una secuencia de uno o varios *plays*, ejecutados en orden, cada uno con su propio grupo de destino. Esa es la unidad que permite que un mismo fichero configure máquinas distintas de forma distinta, y la que Ansible emplea también al informar del resultado, ya que el resumen que imprime al terminar se organiza por máquina a lo largo de todos los *plays* ejecutados.

En este proyecto, el código de Ansible se organiza en roles: una estructura de directorios normalizada (*tasks/* para las tareas, *defaults/* para los valores por defecto de sus variables, *templates/* para las plantillas de configuración) que agrupa todo lo necesario para desplegar un componente lógico completo, y que resulta más mantenible que un único *playbook* plano con una lista larga de tareas. Por último, las plantillas en Ansible se procesan con **Jinja2** [14], un motor de plantillas de Python que permite generar los ficheros de configuración finales de los servicios desplegados a partir de variables y de estructuras condicionales evaluadas en el momento del aprovisionamiento, de modo que una misma plantilla puede generar una configuración distinta según el nodo sobre el que se aplique.

3.4 Docker

Docker [15] es una plataforma de contenedores: permite empaquetar una aplicación junto con todas sus dependencias en una unidad aislada (el contenedor) que comparte el núcleo del sistema operativo anfitrión, a diferencia de una máquina virtual completa, que virtualiza también el propio sistema operativo. Esto hace que los contenedores arranquen mucho más rápido y consuman considerablemente menos recursos que una máquina virtual equivalente. En este proyecto, todo el software del stack LGTM y de Grafana Alloy se despliega como contenedores Docker, gestionados desde Ansible de dos maneras: el agente Grafana Alloy mediante el módulo *docker_container*, y el stack LGTM mediante Docker Compose (la orden `docker compose up` sobre un fichero de composición generado a partir de una plantilla), según se detallará en el capítulo 6. La excepción es MinIO, que se instala como paquete nativo del sistema operativo y se gestiona como servicio de `systemd`, por las razones que se justifican en la sección 5.2.3: su modo distribuido exige que cada nodo sea alcanzable por sus pares bajo la misma dirección y puerto con los que se anuncia, y que disponga de acceso directo al disco que aporta al clúster, condiciones que el aislamiento de red y de sistema

de ficheros del contenedor rompe.

3.5 Docker Compose

Docker Compose [16] es la herramienta de orquestación de contenedores de Docker para un único host: permite definir en un solo fichero declarativo (`docker-compose.yml`, en formato YAML) un conjunto de servicios, cada uno con su imagen, puertos, volúmenes, variables de entorno y dependencias entre sí, y gestionar su ciclo de vida de forma conjunta con una única orden (`docker compose up`). El listado 3.3 recoge un fichero de composición sencillo, con dos servicios, que ilustra ese formato.

```
1 services:
2   web:
3     image: nginx:1.27
4     ports:
5       - "8080:80"          # puerto del anfitrión : puerto del
                             contenedor
6     volumes:
7       - ./sitio:/usr/share/nginx/html
8     depends_on:
9       - db                 # arranca después de la base de datos
10
11   db:
12     image: postgres:16
13     environment:
14       POSTGRES_PASSWORD: ejemplo
15     volumes:
16       - datos-db:/var/lib/postgresql/data
17
18 volumes:
19   datos-db:
```

Listado 3.3: `docker-compose.yml` de ejemplo: una aplicación web y su base de datos

Cada entrada bajo `services` describe un contenedor: la imagen de la que parte, los puertos que publica en el anfitrión, los volúmenes que monta para conservar datos o inyectar ficheros, las variables de entorno con las que se configura y sus dependencias con el resto. A diferencia de un orquestador de clúster como Kubernetes, no coordina contenedores entre varias máquinas, lo que lo hace adecuado justamente para el caso de este proyecto: agrupaciones de contenedores fuertemente acopladas que conviven en un mismo nodo. Se emplea en dos puntos: la orquestación del stack LGTM completo, cuyo fichero de composición se genera desde una plantilla Jinja2 durante el aprovisionamiento, y el despliegue de la [aplicación de](#)

demostración de Grafana Labs (The New Stack) (TNS), compuesta por tres servicios (base de datos, aplicación y generador de carga), en los nodos de aplicación del entorno.

3.6 MinIO

MinIO [4] es un sistema de almacenamiento de objetos de código abierto, compatible con la API de S3, escrito en Go y distribuido como un único binario. En su modo distribuido, que es el empleado en este trabajo, varios nodos se agrupan en un único espacio de nombres lógico y los datos se protegen mediante codificación por borrado (*erasure coding*) [17] en lugar de réplicas completas: cada objeto se divide en fragmentos de datos más fragmentos de paridad calculados matemáticamente a partir de ellos, de forma que el objeto puede reconstruirse aunque se pierda parte de esos fragmentos. Los umbrales de **quórum** que determinan cuándo el clúster admite operaciones se cuentan en discos del conjunto de codificación, y no en nodos: para leer basta con reunir los k fragmentos de datos, mientras que para escribir MinIO exige siempre más de la mitad de los discos del conjunto. Cuando la paridad equivale exactamente a la mitad del conjunto, que es el caso de este despliegue, MinIO eleva además el umbral de escritura en una unidad para evitar situaciones de *split-brain*, de modo que las lecturas pasan a tolerar una pérdida de discos que las escrituras ya no admiten. Esta asimetría tiene implicaciones prácticas directas sobre el comportamiento del clúster ante la caída de un nodo, tal y como se comprobará de forma experimental en la sección 7.3.

En este TFG, MinIO se despliega en modo distribuido entre los dos nodos de almacenamiento del entorno, y actúa además como *backend* de almacenamiento de objetos compatible con S3 para los tres componentes de persistencia del stack LGTM (Loki, Tempo y Mimir), además de exponer su propio extremo de métricas en formato Prometheus, consumido a su vez por el agente de recolección del stack.

3.6.1 Redundancia: réplicas, RAID y codificación por borrado

Cualquier sistema de almacenamiento distribuido necesita algún mecanismo que permita seguir sirviendo los datos cuando un disco o un nodo dejan de estar disponibles. Las tres familias de técnicas habituales son las siguientes.

La **replicación** consiste sencillamente en guardar n copias completas e idénticas de cada objeto en dispositivos o nodos distintos. Es la técnica más simple de implementar y de razonar, y la que ofrece lecturas más baratas, puesto que cualquier copia sirve por sí sola y no hay que reconstruir nada. Su inconveniente es el coste de capacidad: tolerar la pérdida de dos copias exige almacenar tres, es decir, un 200 % de sobrecoste de espacio. Es el modelo que emplean por defecto sistemas como Garage o SeaweedFS, dos de las alternativas a MinIO comparadas en el capítulo 2.

El **RAID** clásico (*Redundant Array of Independent Disks*) agrupa varios discos de una misma máquina y los presenta como un único dispositivo lógico, con distintos niveles que ofrecen distintos compromisos entre espacio y tolerancia a fallos. En RAID 1 los datos se escriben de forma idéntica en dos discos (espejo), sin ningún cálculo de paridad, con el mismo sobrecoste de espacio del 100 % que una réplica. Los niveles basados en paridad, en cambio, reparten los datos entre los discos añadiendo información redundante calculada a partir de ellos: en RAID 5, un bloque de paridad por cada franja de datos, lo que tolera el fallo de un disco; en RAID 6, dos, lo que tolera el fallo de dos. Estos últimos aprovechan mejor el espacio que la replicación, pero el RAID opera a nivel de un único host y de dispositivos de bloque, por lo que no protege frente a la pérdida de un nodo completo ni encaja de forma natural en un sistema distribuido.

La **codificación por borrado** (*erasure coding*) generaliza la idea de la paridad de RAID a un entorno distribuido y a un número arbitrario de fragmentos de redundancia. MinIO la implementa mediante códigos de **Reed-Solomon** [17]: cada objeto se divide en k fragmentos de datos (*data shards*) y, a partir de ellos, se calculan matemáticamente m fragmentos de paridad (*parity shards*); los $k + m$ fragmentos resultantes se distribuyen entre los discos del conjunto de codificación (*erasure set*), procurando repartirlos entre nodos distintos. La propiedad que hace útil este esquema es que cualesquiera k de los $k + m$ fragmentos bastan para reconstruir el objeto original, sea cual sea la combinación concreta que sobreviva: el sistema tolera, por tanto, la pérdida de hasta m fragmentos. MinIO denota esta configuración como $EC:m$, indicando el número de fragmentos de paridad.

La ventaja frente a la replicación es de eficiencia de espacio. Un esquema $k = 6, m = 2$ tolera dos fallos simultáneos con un sobrecoste de solo el 33 % ($8/6$), mientras que conseguir esa misma tolerancia por replicación exigiría tres copias completas, es decir, un 200 %. El precio a pagar es computacional y de latencia: escribir obliga a calcular los fragmentos de paridad, y leer un objeto del que falta algún fragmento obliga a reconstruirlo resolviendo el sistema de ecuaciones del código, en lugar de limitarse a leer una copia intacta. Es un compromiso que resulta claramente favorable en almacenamiento de objetos, donde el volumen de datos es grande y el coste de capacidad domina.

Los valores concretos de k y m que MinIO escoge no son libres: dependen del número de discos del conjunto de codificación, y de ellos se derivan directamente los umbrales de **quórum** que determinan cuándo el clúster admite lecturas y escrituras. La sección 5.5.1 desarrolla esta aritmética para el caso concreto de los dos nodos de este despliegue, y la sección 7.3 verifica experimentalmente el comportamiento que de ella se deduce.

3.7 El stack LGTM

Todo el stack trabaja sobre telemetría: los datos que un sistema emite sobre su propio funcionamiento. Esa telemetría se materializa en los tres tipos de señal definidos en la sección 2.2.1: métricas, logs y trazas.

La correlación entre esas tres señales, que es lo que distingue a una plataforma de observabilidad de la monitorización tradicional, se apoya en su carácter complementario [18]: una métrica revela la anomalía, las trazas de ese intervalo localizan el servicio responsable y sus logs explican la causa concreta. Es el flujo de diagnóstico que se valida en el capítulo 7: de un panel de métricas a una traza, y de esta a sus logs correlacionados por `traceID`. Cada componente del stack LGTM está especializado en almacenar y consultar una de estas señales, con Grafana como capa común de consulta y visualización sobre todas ellas.

3.7.1 Loki

Loki [19] es el componente del stack responsable de logs. A diferencia de los sistemas de gestión de logs basados en indexación de texto completo, no construye un índice invertido (la estructura, habitual en los motores de búsqueda, que asocia cada término del texto con las entradas que lo contienen) sobre el contenido completo de cada línea de log, sino únicamente sobre un conjunto reducido de etiquetas (*labels*), de forma similar a como Prometheus indexa series de métricas; el contenido textual se comprime y se almacena en bloques sobre el *backend* de objetos (MinIO), y solo se recorre en el momento de una consulta ya acotada por las etiquetas indexadas. Las consultas se expresan en **LogQL**. En este proyecto, Loki recibe tanto los logs de los contenedores Docker como los del propio sistema operativo de cada nodo.

3.7.2 Mimir

Mimir [20] es el componente responsable de las métricas, compatible con el protocolo de *remote-write* de Prometheus: los agentes de recolección empujan activamente las métricas hacia Mimir (modelo *push*), en lugar de que sea Mimir quien las recoja mediante *scraping* directo (el modelo *pull* clásico de Prometheus, en el que es el servidor quien consulta periódicamente un extremo HTTP en el que cada servicio expone sus métricas).

El *remote-write* merece un desarrollo aparte, por ser la pieza que conecta la recolección con el almacenamiento en todo el *pipeline* de métricas de este proyecto. Es un protocolo unidireccional y sencillo: el emisor acumula en memoria las muestras que va recogiendo y, de forma periódica, las envía en lotes mediante una petición HTTP POST al extremo de escritura del receptor. El cuerpo de la petición es una estructura serializada en Protocol Buffers y comprimida con Snappy, en la que cada elemento asocia el conjunto de etiquetas que identifica una serie temporal con la lista de pares (marca de tiempo, valor) observados desde el envío

anterior. El receptor responde confirmando la escritura o rechazando el lote, y el emisor mantiene una cola con reintentos para no perder muestras ante una indisponibilidad transitoria del receptor.

La razón de ser de este modelo es que el *pull* clásico no escala bien cuando el almacenamiento es central y las fuentes están distribuidas: obligaría al servidor a conocer, alcanzar por red y sondear individualmente cada extremo de métricas de cada nodo. Con *remote-write*, la responsabilidad se invierte y basta con que cada agente conozca una única dirección, la del receptor. En este proyecto conviven, por eso, los dos modelos: Grafana Alloy hace *pull* en local, sobre los extremos de métricas de su propio nodo (donde la red no es un problema), y hace *push* hacia Mimir por *remote-write* para atravesar la red del laboratorio una sola vez y con un único destino. El mismo mecanismo lo reutiliza internamente el *metrics-generator* de Tempo para publicar en Mimir las métricas que deriva de los *spans*, una pieza clave de la correlación entre señales cuya configuración concreta se describirá en la sección 6.2.6.

Internamente, Mimir organiza las series en bloques con el mismo formato que la [Time Series Database \(base de datos de series temporales\) \(TSDB\)](#) nativa de Prometheus, volcados de forma periódica al *backend* de objetos. Las consultas se expresan en **PromQL**, el lenguaje de consulta de Prometheus. En este proyecto, Mimir recibe tanto las métricas de sistema como las de la aplicación TNS y las del propio clúster MinIO.

3.7.3 Tempo

Tempo [21] es el componente responsable de las trazas distribuidas, apoyado principalmente en la búsqueda directa por identificador de traza (`traceID`) en lugar de en un índice de búsqueda extenso sobre atributos, lo que reduce su coste de ingesta y almacenamiento. Las consultas más exploratorias se expresan en **TraceQL**. Tempo admite trazas en varios formatos de entrada gracias a sus distintos *receivers*: el formato nativo [OpenTelemetry Protocol \(OTLP\)](#), y formatos de compatibilidad con sistemas de trazado anteriores como Jaeger. En este trabajo, esa compatibilidad con Jaeger es la que permite que la [TNS \[22\]](#), instrumentada originalmente para Jaeger, envíe sus trazas directamente al receptor Jaeger nativo de Tempo sin necesidad de reinstrumentarla, una decisión de diseño que se justificará en la sección 5.3.1.

3.7.4 Grafana

Grafana actúa como capa unificada de consulta y visualización sobre los tres componentes anteriores, mediante un modelo de *datasources* (una por *backend*) que permite correlacionar señales de distinto tipo dentro de un mismo panel: saltar, por ejemplo, de una traza en Tempo a los logs de Loki del mismo intervalo temporal. En este proyecto, Grafana se despliega con sus *datasources* y varios paneles (*dashboards*) aprovisionados automáticamente, sin intervención manual. La figura 3.2 recoge uno de ellos, el del clúster de MinIO, con datos reales del entorno

desplegado: reúne el estado del clúster, la capacidad ocupada, el número de *buckets* y de objetos y las tasas de peticiones y de transferencia de la API S3, todo ello a partir de las métricas que Alloy recolecta del *endpoint* Prometheus de MinIO y envía a Mimir.



Figura 3.2: Panel del clúster de MinIO en Grafana, aprovisionado automáticamente durante el despliegue

3.7.5 Lenguajes de consulta: PromQL, LogQL y TraceQL

Cada uno de los tres almacenes del stack expone su propio lenguaje de consulta, todos ellos declarativos y accesibles desde Grafana sobre la *datasource* correspondiente. Pese a estar especializados en un tipo de señal distinto, comparten un rasgo de diseño común que resulta clave para la observabilidad: la selección inicial de datos se apoya en los mismos metadatos indexados (etiquetas o atributos) sobre los que cada componente construye su índice, y no en un barrido de texto completo.

PromQL es el lenguaje de consulta de Prometheus, empleado por Mimir para las métricas. Selecciona series temporales por nombre de métrica y etiquetas (`{job="tns/app"}`) y permite aplicar funciones sobre rangos temporales (`rate`, `increase`) y operadores de agregación (`sum`, `avg`) agrupando por etiquetas, de modo que a partir de contadores en bruto se obtienen tasas y agregados listos para representar en un panel.

LogQL, el lenguaje de Loki, reutiliza deliberadamente el modelo de etiquetas de PromQL: toda consulta parte de un selector de flujos de log (`{container="tns-app"}`) al que se encadena, de forma opcional, una tubería (*pipeline*) de filtros y analizadores (*parsers*) que operan ya sobre el contenido textual de las líneas seleccionadas. LogQL permite además derivar métricas a partir de los propios logs (`count_over_time`, `rate`), tendiendo un puente entre las

dos señales.

TraceQL es el lenguaje de consulta de Tempo para exploración de trazas. En lugar de partir de un `traceID` concreto, permite localizar trazas mediante condiciones estructurales y de atributos sobre sus *spans* (duración, nombre del servicio, atributos de recurso o de *span*); por ejemplo, trazas con algún *span* cuya duración supere un umbral en un servicio dado.

Esta raíz común, seleccionar por metadatos indexados, es precisamente lo que hace posible la correlación entre señales dentro de Grafana: una etiqueta o atributo compartido, como el `traceID` presente tanto en una traza de Tempo como en sus logs de Loki, permite saltar de una señal a otra sin salir de la interfaz. Este flujo de diagnóstico se valida de forma práctica en el capítulo 7.

3.7.6 Grafana Alloy

Grafana Alloy [9] es el agente unificado de recolección: sustituye a la combinación históricamente más habitual de varios agentes independientes (por ejemplo, Prometheus Node Exporter para métricas y Promtail para logs) por un único proceso. Su configuración se define en la **sintaxis de configuración de Alloy** (heredera del lenguaje River de Grafana Agent), un lenguaje declarativo propio de Grafana Labs en el que cada bloque instancia un componente con entradas y salidas explícitas, formando un grafo de flujo de datos. En este proyecto, Alloy se despliega en los nodos de aplicación del entorno y recolecta tres tipos de datos: las métricas básicas del sistema operativo (CPU, memoria, disco, red), mediante su componente `prometheus.exporter.unix`, que integra el exportador estándar **Node Exporter** de Prometheus y evita así desplegarlo como proceso independiente; las métricas de la aplicación TNS y del propio clúster MinIO; y los logs tanto de los contenedores Docker como de sistema. Actúa además como receptor OTLP genérico que reenvía a Tempo.

3.8 OpenTelemetry

OpenTelemetry [10] es el estándar abierto de instrumentación que se ha consolidado en los últimos años como formato común para el transporte de telemetría entre aplicaciones y plataformas de observabilidad, independientemente del proveedor final. Define tanto un conjunto de convenciones semánticas para nombrar de forma consistente atributos habituales (el nombre de un servicio, el método HTTP de una petición) como un protocolo de transporte, OTLP, sobre gRPC o HTTP. En este proyecto, Grafana Alloy actúa como receptor OTLP nativo para cualquier aplicación instrumentada de esta forma.

3.9 Herramientas y lenguajes de apoyo

Además de las tecnologías principales ya descritas, el proyecto se apoya en un conjunto de herramientas y lenguajes transversales, que se describen a continuación.

3.9.1 Git

Todo el código del proyecto (el `Vagrantfile`, los *playbooks* y roles de Ansible, las plantillas de configuración y los scripts auxiliares) se versiona en un repositorio Git [23]. En un proyecto basado en el paradigma IaC esto no es un accesorio, sino parte de la propia definición del paradigma: la infraestructura se describe en ficheros de texto versionables, y es Git quien hace efectiva esa propiedad, proporcionando historial completo de cambios, posibilidad de revertir a un estado anterior conocido y trazabilidad de cada decisión de configuración.

3.9.2 Ruby

Ruby es el lenguaje en el que se escribe el `Vagrantfile`. Como ya se ha señalado en la sección 3.2, más que un formato de configuración se trata de un programa Ruby real que Vagrant evalúa al arrancar, y este proyecto aprovecha esa naturaleza programática: las cuatro máquinas virtuales se definen a partir de una única estructura de datos recorrida en un bucle, y la lectura del fichero externo `config.yml` para seleccionar el sistema operativo se implementa igualmente en Ruby.

3.9.3 YAML

YAML [24] es el formato declarativo de serialización de datos empleado de forma transversal en el proyecto: en él se escriben los *playbooks* y roles de Ansible, el fichero global de configuración `config.yml`, los ficheros `docker-compose.yml` de la aplicación TNS y del stack LGTM, y los ficheros de configuración de Loki, Tempo, Mimir y de los *datasources* de Grafana. Su legibilidad y su carácter declarativo lo han convertido en el formato de facto del ecosistema de infraestructura como código.

3.9.4 Bash

Los pasos de automatización que quedan fuera del alcance natural de Vagrant y Ansible se implementan como scripts de Bash: los *provisioners* de tipo `shell` del `Vagrantfile` que instalan los paquetes base de cada máquina, el script `setup_ssh.sh` que genera y distribuye las claves SSH necesarias para el acceso sin contraseña entre nodos (requisito del modelo *agentless* de Ansible descrito en la sección 3.3), y la herramienta de validación automatizada `scripts/test-provision.sh` descrita en la sección 6.4.

Metodología y planificación del trabajo

ESTE capítulo describe cómo se ha organizado el desarrollo del trabajo. Se presenta en primer lugar la metodología seguida y su relación con las prácticas ágiles; a continuación, las cinco fases en que se estructura el proyecto y su planificación temporal, contrastada con la ejecución real; y, por último, el presupuesto estimado, los riesgos identificados y las desviaciones observadas respecto a lo previsto inicialmente.

4.1 Metodología

El desarrollo de este TFG sigue la metodología definida en el anteproyecto: un enfoque **iterativo e incremental**, en el que las funcionalidades se incorporan de forma progresiva. En cada iteración se desarrolla un componente completo de la infraestructura, comenzando por los elementos esenciales (gestión de las máquinas virtuales, su conectividad y el almacenamiento distribuido) para avanzar después hacia la observabilidad completa con el stack LGTM. Cada iteración se cierra únicamente cuando el resultado se ha podido validar de forma empírica, consultando directamente los servicios desplegados y sin dar nunca por válida una configuración por el mero hecho de que la ejecución de las tareas de Ansible terminase sin errores. Este criterio ha resultado especialmente relevante dado que buena parte de los problemas reales de este proyecto, detallados en la sección 6.3, solo se manifiestan en tiempo de ejecución y no en tiempo de aprovisionamiento.

Este enfoque iterativo e incremental se apoya en los principios de las metodologías ágiles y, en concreto, toma como referencia el marco **Scrum** [25], adaptando aquellos de sus elementos que resultan aplicables a un proyecto académico desarrollado por una única persona. Cada iteración funciona de manera análoga a un sprint: parte de un conjunto acotado y priorizado de funcionalidades (un *backlog* derivado de los objetivos y las fases definidos en el

anteproyecto) y concluye con un incremento de la infraestructura verificado y potencialmente entregable. El criterio de cierre descrito más arriba (no dar por terminada una iteración hasta haberla validado empíricamente) cumple el papel de la definición de terminado (*Definition of Done*) de Scrum, mientras que las reuniones de seguimiento con la persona directora del TFG desempeñan una función próxima a la de la revisión de sprint, aportando la realimentación que orienta la priorización de la iteración siguiente.

No se ha aplicado, sin embargo, Scrum en su forma completa. Al tratarse de un TFG individual no existen los roles diferenciados del marco (*Product Owner*, *Scrum Master* y equipo de desarrollo), que recaen todos en la misma persona, ni tienen sentido las ceremonias concebidas para coordinar a un equipo, como las reuniones diarias; la coordinación externa se limita a la interacción periódica con la dirección del trabajo. Scrum se emplea, por tanto, como referencia metodológica que estructura el trabajo en iteraciones con entregables verificables, y no como un proceso de obligado cumplimiento en todos sus detalles.

4.2 Fases principales del trabajo

De acuerdo con el anteproyecto, el trabajo se organiza en cinco fases:

1. Investigación y diseño arquitectónico, dividida en dos subfases:

- Estudio del estado del arte referente a MinIO, al paradigma IaC (Vagrant y Ansible) y al stack LGTM (capítulos 2 y 3).
- Diseño de la arquitectura lógica del sistema, definiendo el dimensionamiento de las máquinas virtuales (capítulo 5).

2. Aprovisionamiento de la infraestructura base, dividida en dos subfases:

- Despliegue inicial de la infraestructura virtual base mediante Vagrant.
- Integración de Ansible para automatizar las configuraciones del sistema operativo, con soporte multidistribución (Debian/Rocky/Alma).

3. Implementación de almacenamiento y observabilidad, dividida en las siguientes subfases:

- Despliegue del clúster MinIO en modo distribuido.
- Creación de *buckets* S3 en MinIO como *backend* de telemetría.
- Despliegue de Grafana Alloy para la recopilación de logs, métricas y trazas.
- Integración del stack LGTM completo y configuración del *pipeline* de telemetría entre Alloy, Loki, Tempo, Mimir y MinIO.

- Implementación de paneles (*dashboards*) en Grafana para la visualización de los datos recabados.

4. **Validación y pruebas**, dividida en dos subfases (capítulo 7):

- Despliegue de los servicios de prueba mediante un caso de uso basado en la aplicación TNS.
- Ejecución de pruebas de validación de la infraestructura para comprobar su correcto funcionamiento.

5. **Documentación**: redacción y elaboración de la memoria del TFG.

Las fases 2 y 3 son las que se corresponden con el trabajo de desarrollo propiamente dicho, y se describen con ese mismo desglose en el capítulo 6. Dentro de la fase 2, y de forma transversal a toda ella, se llevó a cabo además una reorganización del código de Ansible desde *playbooks* planos hacia roles reutilizables. No se trata de una fase adicional del anteproyecto, sino de una decisión de implementación, motivada por los problemas de mantenibilidad que se documentan en el capítulo 6.

4.3 Planificación

La planificación temporal se articula en torno a las cinco fases de la sección anterior, cada una de las cuales se ha ejecutado como un sprint en el sentido descrito al presentar la metodología: un bloque de trabajo acotado, con un objetivo propio y un incremento verificable al final. La tabla 4.1 recoge la planificación inicial del proyecto: el periodo previsto para cada uno de los sprints al comienzo del trabajo y el esfuerzo estimado para cada uno de ellos. El total estimado, 295 horas, se corresponde con la dedicación equivalente a un TFG de 12 créditos del [European Credit Transfer and Accumulation System \(ECTS\)](#), del orden de 300 horas de trabajo del estudiante.

El desarrollo real no se ajustó por completo a esa previsión. La figura 4.1 contrasta, para cada sprint, el periodo planificado inicialmente (barra superior, en gris) con el periodo de ejecución real (barra inferior, en rosa), de modo que las desviaciones entre lo previsto y lo efectivamente realizado quedan a la vista. La principal desviación se concentró en el sprint 3 (implementación del almacenamiento y la observabilidad): la portabilidad del aprovisionamiento a la familia RHEL (Rocky Linux y AlmaLinux) obligó a resolver una serie de incompatibilidades no previstas (`firewalld` y `nftables`, la ruta de instalación de los binarios, el `secure_path` de `sudo` y el anclado de `requests` para el SDK de Docker, detalladas en el capítulo 6), lo que alargó la fase de implementación más de lo estimado. La segunda desviación afecta al sprint 5 (documentación): al coincidir el tramo final del proyecto con el periodo vacacional del director

Sprint	Actividad	Periodo	Horas
1	Investigación y diseño arquitectónico	19 feb - 5 mar	40
2	Aprovisionamiento de la infraestructura base	9 mar - 10 abr	55
3	Implementación de almacenamiento y observabilidad	13 abr - 12 jun	95
4	Validación y pruebas	15 jun - 17 jul	60
5	Documentación	20 jul - 21 ago	45
Total			295

Tabla 4.1: Planificación temporal y esfuerzo estimado del proyecto (2026)

en agosto, la redacción de la memoria se adelantó y se solapó con las fases de implementación y validación, en lugar de abordarse de forma consecutiva al final, de modo que fuera posible recibir sus revisiones antes de dicho periodo. El sobre coste del sprint 3 se propagó al sprint 4, cuya validación se mantuvo con su duración completa en lugar de recortarla para recuperar el retraso, desplazando el cierre real del proyecto a comienzos de septiembre (en torno al 5 de septiembre, frente al 21 de agosto previsto inicialmente). Esta desviación, de unas dos semanas sobre la planificación inicial, quedó absorbida por el margen disponible hasta la fecha de entrega del trabajo, fijada para el 10 de septiembre.

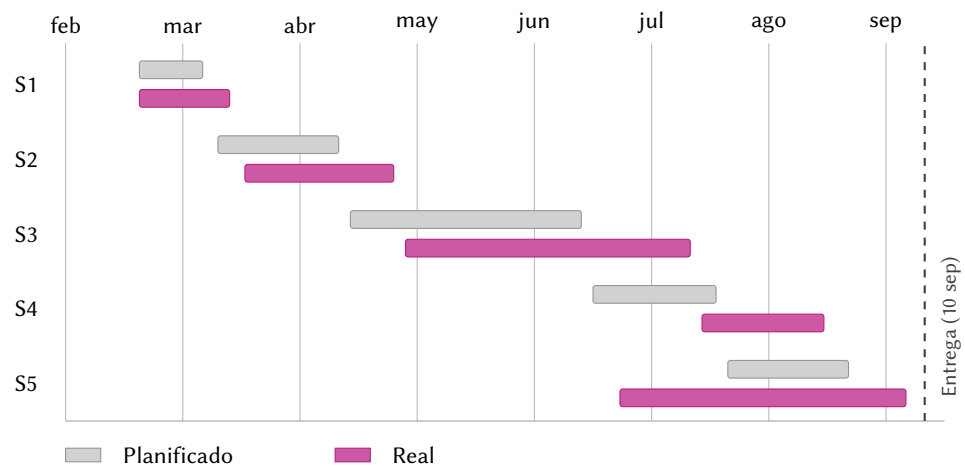


Figura 4.1: Diagrama de Gantt: planificación inicial frente a ejecución real

Ese desplazamiento del calendario vino acompañado de un mayor esfuerzo dedicado al proyecto. La tabla 4.2 contrasta, sprint a sprint, las horas estimadas inicialmente con las real-

mente invertidas. El total real asciende a 358 horas, algo más de un 21 % por encima de las 295 estimadas, y algo más de dos tercios de esa diferencia se concentran en el sprint 3 por los motivos ya descritos. Esta cifra corregida, y no la estimación inicial, es la que se emplea como base del presupuesto en la sección siguiente.

Sprint	Actividad	Estimadas	Reales	Desviación
1	Investigación y diseño arquitectónico	40	44	+4
2	Aprovisionamiento de la infraestructura base	55	63	+8
3	Implementación de almacenamiento y observabilidad	95	138	+43
4	Validación y pruebas	60	62	+2
5	Documentación	45	51	+6
Total		295	358	+63

Tabla 4.2: Esfuerzo estimado frente a esfuerzo real, en horas, por sprint

4.4 Presupuesto

Al tratarse de un proyecto académico de laboratorio, construido íntegramente sobre software de código abierto y ejecutado sobre el equipo personal de desarrollo, el coste del proyecto se concentra casi por completo en el esfuerzo de personal: el gasto en licencias de software es nulo y no existe coste de infraestructura en la nube, al ejecutarse las cuatro máquinas virtuales en local. Los importes de la tabla 4.3 son, en consecuencia, una estimación orientativa basada en los supuestos que se detallan a continuación.

La partida de personal se calcula a partir del esfuerzo realmente invertido en el trabajo, las 358 horas recogidas en la tabla 4.2, valoradas a la tarifa horaria de un perfil de ingeniería junior. Se toma esa cifra corregida, y no las 295 horas estimadas al planificar el proyecto, porque es la que refleja el coste real de desarrollarlo. La amortización de hardware corresponde al equipo de desarrollo sobre el que se ejecuta todo el laboratorio, repartida de forma lineal a lo largo de su vida útil estimada. El software empleado (VirtualBox, Vagrant, Ansible, MinIO, el stack LGTM, Docker CE y las distribuciones Linux) es de código abierto y no genera coste de licencia.

Los dos supuestos con mayor impacto sobre el total, la tarifa horaria (20 €/h) y el coste del equipo de desarrollo (1.000 €), pueden ajustarse a los valores reales sin alterar la estructura del cálculo. La estimación no incluye impuestos (IVA) ni beneficio industrial, al no tratarse de un presupuesto destinado a facturación.

Concepto	Base de cálculo	Coste (€)
Personal (desarrollo)	358 h $\times$ 20 €/h	7.160
Amortización de hardware	Equipo de desarrollo ($\approx$ 1.000 €, vida útil 4 años, $\approx$ 6,5 meses de uso)	135
Licencias de software	Herramientas de código abierto	0
Costes indirectos	Electricidad y conectividad	150
Total		7.445

Tabla 4.3: Estimación de presupuesto del proyecto

4.5 Análisis de riesgos

Durante la ejecución del proyecto se han identificado y materializado los siguientes riesgos, con su correspondiente mitigación:

- **Sobreasignación de recursos del host.** Las cuatro máquinas virtuales, en su asignación actual de memoria, ocupan la totalidad de la RAM disponible en el equipo de desarrollo utilizado. Este riesgo se materializó en la práctica durante la fase 3: uno de los nodos del clúster (minio-2) llegó a quedar inaccesible por inestabilidad de la máquina virtual, requiriendo un reinicio completo. La mitigación aplicada de forma inmediata fue el reinicio y reaprovisionamiento del nodo afectado, y de forma estructural, el ajuste a la baja de la asignación de memoria de las cuatro máquinas hasta dejar margen libre en el equipo anfitrión.
- **Incompatibilidades no documentadas entre sistemas operativos.** El soporte multidistribución era un requisito explícito de la fase 2, pero el alcance real de las incompatibilidades entre Debian, Rocky Linux y AlmaLinux no era conocido de antemano. Este riesgo se materializó de forma repetida durante las fases 2 y 3 (sección 6.3), con un esfuerzo de diagnóstico y corrección considerablemente mayor al previsto inicialmente. La mitigación seguida ha sido sistemática: diagnóstico basado en evidencia directa del sistema (registros, herramientas de bajo nivel) en lugar de asumir la causa del fallo, y verificación explícita, para cada corrección aplicada, de si el mismo problema también afectaba a las otras dos distribuciones o solo a aquella en la que se detectó.
- **Ajuste al plazo de entrega.** Con la persona directora del TFG no disponible durante el mes de agosto, existía el riesgo de no disponer de una ronda de revisión antes del plazo límite de entrega. La mitigación aplicada fue adelantar la redacción de una primera versión completa de la memoria (fase 5) antes de dicho periodo, de forma que quedase margen para incorporar el feedback recibido antes de la fecha límite.

4.6 Resumen de ejecución y desviaciones

La ejecución del proyecto se ha ajustado, en líneas generales, a las cinco fases planificadas en el anteproyecto, con las siguientes desviaciones respecto a lo previsto inicialmente:

- El esfuerzo dedicado al soporte multidistribución (fase 2) y a su validación sobre cada componente añadido en la fase 3 ha sido considerablemente mayor de lo previsto (43 horas por encima de lo estimado solo en la fase 3, tabla 4.2), al tratarse de una sucesión de problemas que solo se manifestaban en tiempo de ejecución y no durante el propio aprovisionamiento con Ansible.
- El uso de roles de Ansible ya figuraba como objetivo en el anteproyecto. Como primer enfoque, sin embargo, se optó por implementar cada componente con *playbooks* planos y comprobar que funcionaba antes de reorganizar el código en roles. Esa reorganización llevó más tiempo del planificado, y una vez completada los *playbooks* planos quedaron fuera del proyecto (véanse las conclusiones, capítulo 8).
- Tras la planificación inicial, el autor comenzó a trabajar, lo que redujo la dedicación diaria disponible para el proyecto respecto a la prevista y alargó el desarrollo en el calendario. Esta menor dedicación, unida a la no disponibilidad de la dirección del TFG durante el mes de agosto (riesgo recogido en la sección 4.5 anterior), reforzó la decisión de adelantar la redacción de una primera versión completa de la memoria.

Diseño y arquitectura

ESTE capítulo describe las decisiones de diseño que dan forma a la arquitectura del sistema, así como la justificación de cada una de ellas. El capítulo 6 detalla, por su parte, cómo se materializa esta arquitectura mediante Vagrant y Ansible.

5.1 Requisitos

Del anteproyecto se derivan cuatro requisitos principales, que actúan como hilo conductor de todas las decisiones de diseño recogidas en este capítulo:

- Un sistema de almacenamiento de objetos distribuido, tolerante a fallos, desplegado sobre varios nodos.
- Una pila de observabilidad nativa de la nube capaz de recolectar y correlacionar métricas, logs y trazas de forma unificada.
- Un despliegue completamente automatizado y reproducible mediante el paradigma IaC, sin pasos de configuración manual.
- Una aplicación de demostración que genere carga real, para validar que el *pipeline* de observabilidad funciona de extremo a extremo y no solo sobre el papel.

5.2 Arquitectura lógica

La arquitectura resultante se organiza en cuatro nodos, descritos con detalle en la tabla 6.1 del capítulo de implementación, agrupados en dos planos claramente diferenciados. Conviene subrayar que nodo designa aquí un servidor independiente, con su propio sistema operativo, su propio disco y su propia dirección en la red: que en este trabajo esos servidores se materialicen como máquinas virtuales sobre VirtualBox es una consecuencia de no disponer de

hardware físico, tal y como se justifica en la sección 3.1, y no una propiedad de la arquitectura, que sería exactamente la misma sobre cuatro servidores físicos.

- Un **plano de almacenamiento y observabilidad**, formado por minio-1 y minio-2, que alberga tanto el clúster distribuido de MinIO como, en el caso de minio-1, la totalidad del stack LGTM.
- Un **plano de aplicación**, formado por app-monitor-1 y app-monitor-2, que ejecuta la carga de trabajo de demostración (TNS) y el agente de recolección de telemetría (Grafana Alloy).

Esta separación en dos planos no es arbitraria: refleja la separación de responsabilidades entre quien genera telemetría y quien la almacena y la sirve, que es justamente la que cualquier sistema de observabilidad real necesita mantener para no acoplar el ciclo de vida de las aplicaciones monitorizadas al de la propia plataforma de monitorización. La figura 5.1 representa gráficamente esta arquitectura: los cuatro nodos, su agrupación en los dos planos descritos y la red privada que los interconecta.

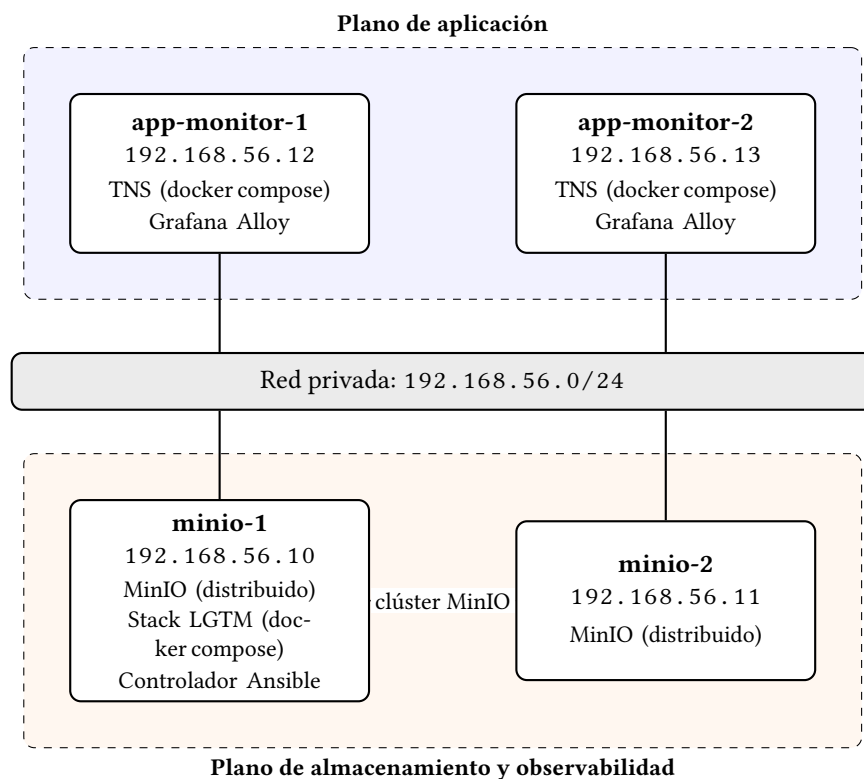


Figura 5.1: Arquitectura lógica del sistema: cuatro nodos agrupados en dos planos e interconectados por una red privada

5.2.1 Elección de `minio-1` como nodo controlador

En lugar de introducir un quinto nodo dedicado exclusivamente a tareas de orquestación, como sería razonable plantear en un entorno de producción, se ha optado por que `minio-1` desempeñe un doble papel: nodo de almacenamiento y controlador Ansible del resto del clúster. Esta decisión reduce el consumo de recursos del entorno, una variable especialmente sensible en un laboratorio que se ejecuta en local sobre VirtualBox con recursos de máquina limitados, a costa de acoplar ligeramente el ciclo de vida del controlador al de un nodo de datos. Se considera un compromiso razonable dado el carácter académico y de laboratorio del proyecto, el mismo que motiva el resto de simplificaciones recogidas en la sección 5.6.

5.2.2 Selección del sistema operativo como parámetro global

Como se detalla en la sección 6.1.1, el sistema operativo de las cuatro máquinas es un único parámetro global, no una decisión por nodo. Se ha optado deliberadamente por esta restricción, en lugar de permitir, por ejemplo, un clúster con nodos de distinto sistema operativo, porque el objetivo del proyecto no es validar la interoperabilidad entre distribuciones distintas dentro de un mismo clúster (un escenario poco realista para un clúster de dos nodos de MinIO, que típicamente se homogeneiza), sino validar que la automatización es portable, en su conjunto, entre Debian, Rocky Linux y AlmaLinux. Un parámetro único, en lugar de tres independientes, simplifica considerablemente tanto el propio `Vagrantfile` como el proceso de prueba descrito en el capítulo 7.

5.2.3 MinIO como servicio nativo del sistema operativo

Todo el software desplegado en este proyecto se ejecuta en contenedores Docker, con una única excepción deliberada: MinIO, que se instala como paquete nativo del sistema operativo (`.deb` o `.rpm`) y se gestiona como un servicio de `systemd`. Es lo que en la terminología del propio proyecto MinIO se denomina un despliegue *bare-metal*, frente al despliegue en contenedor.

Esta no fue la decisión inicial. El primer enfoque adoptado desplegaba también MinIO en contenedores Docker, uno por nodo, por coherencia con el resto del despliegue. Ese enfoque no llegó a funcionar en modo distribuido: los dos contenedores arrancaban correctamente de forma aislada, pero no conseguían formar el clúster entre sí. El modo distribuido de MinIO impone condiciones que el aislamiento del contenedor rompe: cada nodo debe ser alcanzable por los demás mediante un nombre de host estable y bajo el mismo puerto con el que él mismo se anuncia, y debe tener acceso directo y exclusivo al disco que aporta al clúster. En una red *bridge* de Docker, la dirección que cada contenedor ve de sí mismo no coincide con la que sus pares utilizan para alcanzarlo, y esa discrepancia basta para que los nodos no lleguen a formar

el clúster.

Ninguno de los dos requisitos es, en rigor, incompatible con Docker. El acceso al disco se resuelve sin dificultad con un montaje de tipo *bind*, que expone la ruta del anfitrión dentro del contenedor y permite que el proceso lea y escriba en ella como cualquier otro proceso del sistema. El problema de direccionamiento también tiene solución conocida: basta con ejecutar los contenedores de MinIO en modo `network_mode: host`, con lo que cada uno comparte la pila de red de su nodo y se anuncia con la misma dirección con la que sus pares lo alcanzan. Lo que se decidió fue no aplicar esa solución, porque renunciar al aislamiento de red del contenedor deja al despliegue en contenedor sin la propiedad que justificaba usarlo, y una instalación nativa como paquete del sistema operativo consigue lo mismo de forma más directa y con menos piezas móviles.

Instalar MinIO como servicio nativo elimina de raíz las dos condiciones: cada proceso escucha directamente en la IP estática de su nodo (sección 5.4), la misma que aparece en la notación de expansión `http://minio-{1...2}.local:9000/...` con la que se declara el clúster, y monta el disco adicional directamente sobre `/mnt/disk1/minio`. Con ese cambio el clúster distribuido se formó sin ninguna configuración adicional. La decisión tiene además una ventaja de coherencia conceptual con el resto del diseño: MinIO es aquí la capa de almacenamiento persistente sobre la que se apoya todo lo demás, de modo que resulta razonable que sea la única pieza que no dependa de que el demonio de Docker esté operativo.

5.3 Diseño del *pipeline* de observabilidad

El flujo de telemetría atraviesa varios componentes antes de llegar a Grafana, y su diseño responde a una separación clara por tipo de señal:

- Las **métricas** se recolectan mediante `prometheus.exporter.unix` (métricas de sistema) y un *scrape* HTTP directo a los puertos que expone la aplicación TNS, y se remiten a **Mimir** mediante el protocolo de *remote-write* compatible con Prometheus.
- Los **logs** se recolectan tanto de los contenedores Docker en ejecución (vía `discovery.docker`) como de ficheros de sistema en `/var/log`, y se remiten a **Loki**.
- Las **trazas** siguen dos caminos distintos según su origen: las trazas OTLP nativas se reciben en Grafana Alloy y se reenvían a **Tempo**, mientras que las trazas Jaeger de la aplicación TNS se envían directamente desde los contenedores de la aplicación al receptor Jaeger nativo que expone el propio Tempo en `minio-1`, sin pasar por Alloy. Esta última decisión de diseño se justifica con detalle en la sección 5.3.1.

La figura 5.2 resume gráficamente este flujo de telemetría, desde su origen en los nodos `app-monitor-*` hasta su almacenamiento final en MinIO.

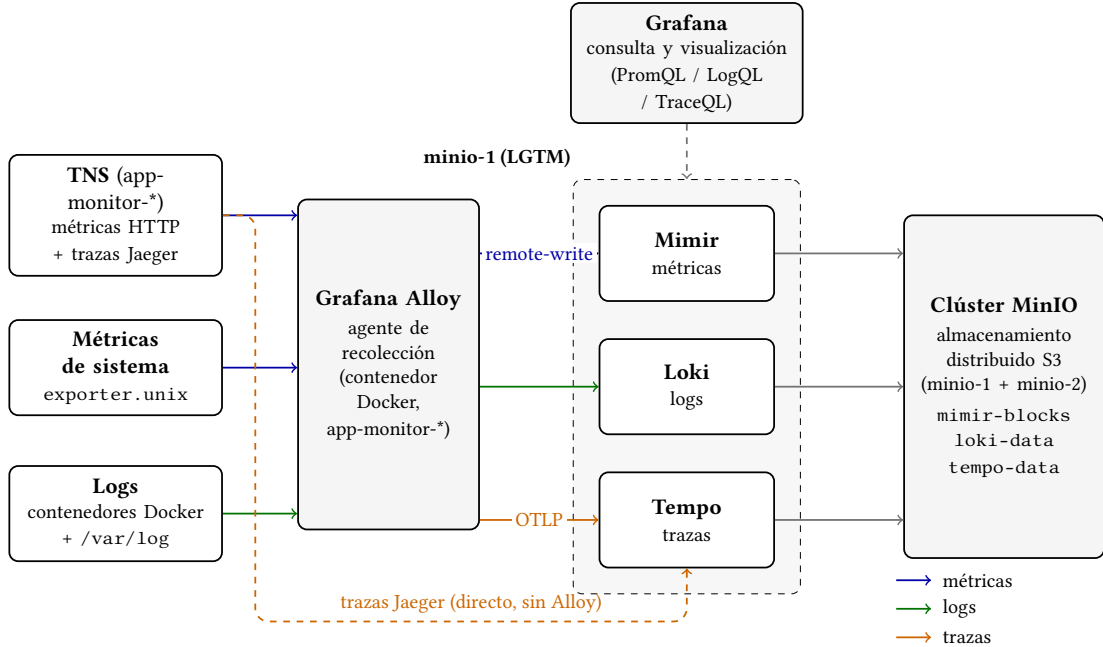


Figura 5.2: Pipeline de telemetría: las tres señales fluyen desde `app-monitor-*` hacia los almacenes LGTM que se ejecutan en `minio-1`, con el clúster MinIO (distribuido entre `minio-1` y `minio-2`) como *backend* S3 y la ruta directa de trazas Jaeger hacia Tempo

En los tres casos, tanto Loki como Mimir como Tempo emplean MinIO como *backend* de almacenamiento de objetos compatible con S3, en lugar de almacenamiento en disco local. El propio sistema bajo estudio (MinIO) es así, a la vez, el objeto de la primera parte del TFG y la infraestructura de persistencia de la segunda, lo que permite validar ambas partes del proyecto con un único flujo de datos real en lugar de dos entornos de prueba independientes.

5.3.1 Justificación de la exclusión de Alloy en las trazas de TNS

Grafana Alloy sí incluye en su configuración un receptor OTLP genérico, pensado para aplicaciones instrumentadas de forma nativa con OpenTelemetry, pero deliberadamente no se ha configurado ningún receptor Jaeger local en el propio Alloy. La aplicación de demostración TNS se despliega mediante `docker-compose.yml` en la red `bridge` por defecto de Docker Compose (una red virtual interna en la que cada contenedor recibe su propia dirección y su propio espacio de red), no en modo `network_mode: host` (en el que el contenedor comparte directamente la pila de red de la máquina anfitriona, como hace aquí el contenedor de Grafana Alloy). En ese escenario, `localhost` dentro de un contenedor de TNS no es la máquina anfitriona sino el propio contenedor, por lo que un receptor Jaeger en el Alloy del

host no sería alcanzable desde dentro de los contenedores de TNS sin recurrir a mecanismos adicionales de resolución de red, como `host.docker.internal` o exponer explícitamente la IP del host. Dado que `docker-compose.yml` ya apunta directamente al receptor Jaeger nativo que expone Tempo en `minio-1`, añadir un salto intermedio por Alloy no aportaría ningún beneficio funcional y sí una complejidad de red innecesaria.

5.4 Diseño de red

Los cuatro nodos comparten una única red privada (`192.168.56.0/24`) con direccionamiento IP estático, en lugar de [Dynamic Host Configuration Protocol \(DHCP\)](#). Esta decisión, habitual en entornos de laboratorio con Vagrant, elimina una fuente de variabilidad entre ejecuciones: todas las referencias entre componentes, como los *endpoints* de Loki, Mimir y Tempo que consume Grafana Alloy o la notación de expansión de nodos de MinIO, pueden codificarse como constantes conocidas de antemano, en lugar de tener que resolverse dinámicamente en tiempo de aprovisionamiento.

En términos de arquitectura, lo que el diseño exige de esa red es únicamente que interconecte los cuatro nodos entre sí, que admita direccionamiento estático y que no quede expuesta al exterior. En un contexto real con servidores físicos, ese requisito se satisfaría con un segmento de red dedicado; en el entorno de laboratorio virtual de este trabajo se materializa como una red *host-only* (en la terminología de Vagrant, una *private_network*), es decir, una red virtual privada que interconecta las cuatro máquinas entre sí y con el anfitrión, pero que no es enrutable hacia el exterior. La conectividad a Internet que necesita el aprovisionamiento (descargar las *boxes* base y los paquetes de cada distribución) la proporciona la interfaz [Network Address Translation \(NAT\)](#) que Vagrant configura por defecto en cada máquina, de modo que ambas funciones quedan separadas en interfaces distintas: la NAT para el tráfico saliente y la *host-only* para el tráfico entre nodos. Se ha optado por el modo *host-only* frente a las dos alternativas habituales por las razones que sostienen el resto del diseño de red. Una red en modo puente (*bridged*) expondría las máquinas, deliberadamente sin endurecer, con credenciales por defecto y sin TLS (sección 5.6), a la red física del anfitrión, y haría que sus direcciones dependieran del DHCP de esa red, perdiendo precisamente la propiedad de direccionamiento fijo y conocido que se acaba de justificar. El modo NAT por sí solo, por su parte, no ofrece conectividad directa y direccionable entre las máquinas, imprescindible para un clúster distribuido. El modo *host-only* es el único que combina aislamiento respecto al exterior, conectividad completa entre nodos y libertad para asignar el direccionamiento estático del subrango `192.168.56.0/24`.

5.5 Alta disponibilidad del clúster MinIO: alcance y limitaciones

El clúster de MinIO se despliega en modo distribuido con dos nodos y codificación por borrado (*erasure coding*). Un clúster de este tamaño impone un compromiso conocido en términos de [High Availability \(alta disponibilidad\) \(HA\)](#), asumido aquí como decisión de diseño y no descubierto a posteriori: MinIO exige [quórum](#) estricto para admitir escrituras, más de la mitad de los discos del conjunto de codificación [17], mientras que para leer basta con reunir los k fragmentos de datos. Con exactamente dos nodos que aportan un disco cada uno, esto se traduce en que el sistema tolera la caída de un nodo para operaciones de lectura, pero no para operaciones de escritura, que quedan bloqueadas hasta que el nodo caído se recupera. El comportamiento observado en la práctica, descrito y verificado experimentalmente en el capítulo 7, confirma lo esperado. Una disponibilidad de escritura tolerante a la caída de un nodo requeriría aumentar el número de dominios de fallo del clúster: manteniendo la configuración de un disco por nodo de este despliegue, harían falta cuatro nodos, que es el mínimo con el que el quórum de escritura sobrevive a la pérdida de un nodo completo. Ampliar el clúster hasta esa cifra queda fuera del alcance de este TFG por las limitaciones de recursos del entorno de laboratorio (véanse las conclusiones, capítulo 8). Los párrafos siguientes precisan de dónde sale ese número, porque la cifra concreta de nodos necesarios depende de cuántos discos aporte cada uno.

Conviene distinguir aquí dos cosas que es fácil confundir. El umbral de quórum se mide en discos, según se acaba de describir, pero la tolerancia a la caída de un nodo depende del número de nodos, y no de cuántos discos tenga cada uno: añadir más discos a los dos nodos existentes no proporcionaría disponibilidad de escritura ante la caída de un nodo. La razón es que un conjunto de codificación se extiende sobre todos los nodos del grupo, de modo que MinIO reparte los fragmentos de cada objeto entre ellos y, con solo dos, cada nodo alberga siempre la mitad de los fragmentos, independientemente de cuántos discos tenga; como el quórum de escritura exige siempre más de la mitad, la pérdida de un nodo deja al clúster exactamente en la mitad, por debajo de dicho umbral. Más discos por nodo solo aportan tolerancia a fallos de discos individuales dentro de un mismo nodo (y más capacidad), pero no convierten en tolerable la caída del nodo completo. La tolerancia a la caída de un nodo en escritura depende, por tanto, del reparto de los discos entre dominios de fallo independientes, y no solo del número total de discos.

Conviene ser preciso con la cifra, porque la afirmación de que hacen falta cuatro nodos solo es cierta bajo el supuesto de un disco por nodo, que es el de este despliegue. En el caso general, un clúster de n nodos con d discos cada uno reúne $N = n \cdot d$ discos, de los que la caída de un nodo se lleva d ; como el quórum de escritura deja margen para perder $N/2 - 1$

discos, la caída de un nodo completo se tolera cuando $n \cdot d/2 - 1 \geq d$, es decir, cuando $d \cdot (n - 2) \geq 2$. Con un disco por nodo ($d = 1$) esa condición exige $n \geq 4$, de donde sale la cifra de cuatro nodos citada más arriba, pero un clúster de tres nodos con dos discos cada uno ($n = 3$, $d = 2$, seis discos en total) también la cumple: la caída de un nodo deja cuatro discos vivos, justo por encima del quórum de escritura. Con dos nodos, en cambio, la condición es imposible de satisfacer para cualquier valor de d , que es la limitación estructural descrita en el párrafo anterior.

La razón de asignar un único disco a cada nodo en este trabajo es de recursos: el entorno se ejecuta sobre un anfitrión de laboratorio, y cada disco adicional supone un fichero de disco virtual más por máquina. Un despliegue de producción seguiría la arquitectura de referencia recomendada por MinIO, de al menos cuatro nodos y varios discos por nodo [4].

5.5.1 Aritmética de fragmentos y quórum para dos nodos

En este despliegue cada nodo aporta un único disco (`/mnt/disk1/minio`), de modo que MinIO forma un único conjunto de codificación (*erasure set*) de $N = 2$ discos. Con dos discos, la única configuración de *erasure coding* posible es **EC:1**, donde cada objeto se codifica mediante Reed–Solomon en $k = 1$ fragmento de datos (*data shard*) y $m = 1$ fragmento de paridad (*parity shard*), y MinIO almacena un fragmento en cada disco, es decir, uno en cada nodo. Al existir un solo fragmento de datos, cualquiera de los dos fragmentos es suficiente para reconstruir el objeto completo, por lo que EC:1 se comporta en la práctica como una réplica espejo.

Sobre esta configuración, MinIO fija dos umbrales de disponibilidad [17]:

- **Quórum de lectura** = $k = 1$. Basta con que un disco (un nodo) esté disponible, ya que un solo fragmento reconstruye el objeto.
- **Quórum de escritura** = $k + 1 = 2$. Cuando el número de fragmentos de paridad iguala al de datos ($m = k$), MinIO incrementa en uno el quórum de escritura para evitar situaciones de *split-brain*; con $N = 2$, esto obliga a que ambos discos (ambos nodos) estén disponibles.

De estos umbrales se deduce directamente la tolerancia a fallos observada: las lecturas toleran la caída de $N - k = 1$ disco, mientras que las escrituras toleran $N - (k + 1) = 0$, es decir, ninguna. Como cada nodo aporta aquí un único disco, perder un disco equivale a perder un nodo completo. Esta es exactamente la asimetría lectura/escritura que se verifica de forma experimental en la sección 7.3.

La misma aritmética explica por qué recuperar la tolerancia a la caída de un disco también en escritura exige, como mínimo, cuatro discos. En el régimen de paridad máxima ($m = k$), el que MinIO aplica a un clúster de dos discos y, por defecto, también a uno de cuatro, el quórum

de escritura vale $k + 1 = N/2 + 1$, de modo que el número de discos que pueden caer sin bloquear las escrituras es $N - (N/2 + 1) = N/2 - 1$. Para que esa cifra alcance 1 se necesita $N \geq 4$: un clúster de cuatro discos (EC:2, con $k = 2$ y $m = 2$) tiene quórum de escritura 3 y tolera, por tanto, la caída de un disco también en escritura. Bajo la premisa de un disco por nodo, ese clúster de cuatro discos es un clúster de cuatro nodos, y la tolerancia expresada en discos se traduce directamente en tolerancia a la caída de un nodo. La tabla 5.1 resume ambos casos.

Discos (N)	EC	k	m	Q. lect.	Q. escr.	Tol. lect.	Tol. escr.
2 (este TFG)	EC:1	1	1	1	2	1	0
4 (referencia)	EC:2	2	2	2	3	2	1

Tabla 5.1: Fragmentos de datos (k) y paridad (m), quórum y tolerancia a la caída de nodos: el clúster de dos discos de este TFG y, como referencia, uno de cuatro discos

5.6 Alcance y seguridad

Por tratarse de un entorno de laboratorio académico, y no de un despliegue de producción, se han tomado decisiones de diseño que deliberadamente priorizan la simplicidad sobre el endurecimiento de la seguridad: credenciales por defecto en texto plano, ausencia de TLS entre componentes, y desactivación completa de `firewalld` en lugar de una apertura selectiva de puertos. Estas decisiones se documentan de forma explícita, precisamente para dejar constancia de que se trata de una limitación de alcance consciente y no de un descuido.

Implementación

EN este capítulo se describe cómo se ha materializado, mediante herramientas IaC, la arquitectura presentada en el capítulo 5. Su estructura reproduce el desglose en fases del anteproyecto recogido en la sección 4.2, del que aquí se desarrollan las dos que implican trabajo de implementación: primero el aprovisionamiento de la infraestructura base con Vagrant y Ansible (fase 2), después la implementación del almacenamiento distribuido y la observabilidad completa (fase 3). Cierra el capítulo una sección transversal a ambas fases con los problemas de compatibilidad entre sistemas operativos surgidos durante el desarrollo, junto con la solución aplicada a cada uno.

6.1 Aprovisionamiento de la infraestructura base

Esta primera fase deja el entorno listo para desplegar servicios sobre él. Las dos subsecciones siguientes recorren sus dos mitades: primero, la definición de las cuatro máquinas virtuales y su aprovisionamiento base con Vagrant; después, la automatización de la configuración con Ansible y su reorganización en roles reutilizables.

6.1.1 Aprovisionamiento con Vagrant

El entorno completo se levanta sobre cuatro máquinas virtuales de VirtualBox, orquestadas mediante un único Vagrantfile. Las cuatro comparten una red privada (192.168.56.0/24) y difieren únicamente en el rol que desempeñan y en los recursos que se les asignan, tal y como resume la tabla 6.1.

Una de las decisiones de diseño más relevantes en esta capa es que el sistema operativo de las cuatro máquinas se selecciona una única vez, de forma global, en un fichero de configuración externo (config.yml) que el propio Vagrantfile lee al arrancar:

```
1 sistema_operativo: debian # opciones: debian | rocky | alma
```

Listado 6.1: Selección de sistema operativo en config.yml

Máquina	IP	RAM	vCPU	Rol	Servicios
minio-1	192.168.56.10	3 GB	2	Controlador	Ansible, MinIO, LGTM
minio-2	192.168.56.11	3 GB	2	Worker	MinIO
app-monitor-1	192.168.56.12	1 GB	2	Worker	App TNS, Grafana Alloy
app-monitor-2	192.168.56.13	1 GB	2	Worker	App TNS, Grafana Alloy

Tabla 6.1: Topología de las cuatro máquinas virtuales del entorno

Esto permite reproducir el despliegue completo sobre Debian, Rocky Linux o AlmaLinux sin modificar ni una línea del `Vagrantfile` ni de los *playbooks* de Ansible, algo especialmente útil de cara a la validación de compatibilidad multi-sistema operativo descrita en la sección 6.3. El listado 6.2 recoge el extracto del `Vagrantfile` sobre el que se apoya el resto de esta subsección, y en él pueden seguirse una por una las decisiones que se describen a continuación.

Las cuatro máquinas no se declaran de forma independiente, sino que se derivan de una única estructura de datos, `MACHINES` (líneas 1-17), que el bucle de las líneas 27-28 recorre para definir cada una con su dirección, su memoria y su número de procesadores virtuales. Añadir un nodo al entorno se reduce, por tanto, a añadir una entrada más a esa estructura. Además de las tareas de aprovisionamiento base (instalación de paquetes mínimos, habilitación de SSH), a los nodos `minio-1` y `minio-2` se les configura un disco virtual adicional destinado al almacenamiento distribuido de MinIO, que es el bloque condicional de las líneas 35-38, y en `minio-1` se instala además Ansible, ya que actúa como controlador del resto del clúster.

Los *playbooks* se ejecutan directamente desde la carpeta sincronizada entre el anfitrión y las máquinas virtuales (`/vagrant/ansible`), que es la declarada en las líneas 23-25, de modo que cualquier edición del código de aprovisionamiento en el equipo anfitrión es visible en el controlador de forma inmediata, sin necesidad de reaprovisionar. Los permisos restringidos que fija esa misma declaración (`dmode=0755`, `fmode=0644`, línea 25) no son un detalle cosmético, y de ahí el comentario que los acompaña en el propio fichero: sin ellos VirtualBox monta la carpeta como escribible por cualquier usuario, y Ansible se niega a cargar su fichero de configuración (`ansible.cfg`) desde un directorio en esas condiciones.

```

1 MACHINES = {
2   'minio-1' => {
3     'ip' => '192.168.56.10',
4     'memory' => 3072,
5     'cpus' => 2,

```

```

6   'role' => 'controller',
7   'description' => 'Controladora con Ansible y MinIO'
8 },
9 'minio-2' => {
10   'ip' => '192.168.56.11',
11   'memory' => 3072,
12   'cpus' => 2,
13   'role' => 'worker',
14   'description' => 'Worker con MinIO'
15 },
16 # ... app-monitor-1/2 (192.168.56.12/13), 1024 MB, rol 'worker'
17 }
18 Vagrant.configure("2") do |config|
19   config.vm.box = SELECTED_BOX
20   config.vm.box_version = SELECTED_VERSION
21   # Carpeta sincronizada con permisos restringidos: VirtualBox monta
22   # /vagrant world-writable y Ansible rechaza leer ansible.cfg así
23   config.vm.synced_folder ".", "/vagrant",
24     owner: "vagrant", group: "vagrant",
25     mount_options: ["dmode=0755", "fmode=0644"]
26
27   MACHINES.each do |name, machine_config|
28     config.vm.define name do |machine|
29       machine.vm.network "private_network", ip: machine_config['ip']
30       machine.vm.hostname = name
31       machine.vm.provider "virtualbox" do |vb|
32         vb.name = name
33         vb.memory = machine_config['memory']
34         vb.cpus = machine_config['cpus']
35         # Disco adicional (10 GB) solo en los nodos MinIO
36         if machine_config['role'] == 'controller' || name == 'minio-2'
37           # ... createmedium + storageattach ...
38         end
39       end
40       # ... shell: paquetes base y, en la controladora, Ansible ...
41     end
42   end
43 end

```

Listado 6.2: Extracto del Vagrantfile: las cuatro máquinas desde MACHINES

6.1.2 Automatización con Ansible: de *playbooks* a roles

La configuración software de las cuatro máquinas se gestiona íntegramente con Ansible. El proyecto comenzó con una primera implementación formada por cuatro *playbooks* planos (los ficheros `playbook-*.yaml`: `docker`, `minio`, `grafana-alloy` y `lgdm-stack`), cada uno con una única lista larga de tareas. Sobre esa base se llevó a cabo una refactorización completa hacia roles de Ansible, que constituye uno de los objetivos explícitos de este TFG (véase capítulo 1) y que sustituyó por completo a los *playbooks* planos: desde ese momento, la implementación basada en roles es la única que forma parte del proyecto, y es la que se describe en el resto de esta sección.

El orden de esa secuencia, primero *playbooks* planos y solo después roles, fue deliberado. Escribir las tareas seguidas en un único fichero es más simple y más ágil mientras se está averiguando qué hay que hacer exactamente en cada nodo, porque no obliga a decidir de antemano cómo se reparten las responsabilidades ni a mantener una estructura de directorios que todavía va a cambiar. Una vez que las tareas estaban ya funcionando y verificadas, la organización correcta dejó de ser una hipótesis y pasó a deducirse del código escrito, momento en el que se acometió la migración a roles.

Criterio de reparto de responsabilidades

El reparto de responsabilidades entre roles se ha hecho siguiendo un criterio de cohesión funcional: cada rol agrupa las tareas necesarias para desplegar un componente lógico completo del sistema, no una tecnología aislada. La tabla 6.2 resume los cinco roles resultantes.

Rol	Responsabilidad	Hosts
<code>docker</code>	Instalación de Docker y Docker Compose	todos
<code>firewall</code>	Neutralización de <code>firewalld</code> en Rocky/Alma	minio, monitoring
<code>minio</code>	Despliegue del clúster distribuido de MinIO	minio-1, minio-2
<code>grafana_alloy</code>	Despliegue del agente Grafana Alloy	app-monitor-1/2
<code>lgdm_stack</code>	Despliegue de Loki, Grafana, Tempo y Mimir	minio-1

Tabla 6.2: Roles de Ansible y su responsabilidad

El rol `firewall` merece una mención aparte: no se creó por tener una responsabilidad de negocio propia, sino porque la lógica de arranque y desactivación de `firewalld` en

Rocky/Alma (descrita en detalle en la sección 6.3) estaba copiada de forma idéntica en los tres *playbooks* que la necesitaban. Extraerla a un rol permite definirla una única vez y reutilizarla como dependencia de los roles `minio`, `grafana_alloy` y `lgtm_stack`.

Esa centralización resultó además ser algo más que una limpieza estética. La maniobra de arrancar y parar `firewalld` modifica el estado del sistema por definición, así que repetirla en cada ejecución rompía la *idempotencia* del despliegue completo en Rocky y Alma. Al vivir en un único sitio, la condición que decide si la maniobra hace falta (comprobar si el servicio está activo, si arrancaría en el próximo inicio o si ha dejado una tabla `nftables` huérfana) se implementa una sola vez y beneficia a los tres *plays* que usan el rol. El análisis completo del problema y su medición están en la sección 7.4.

El listado 6.3 muestra la estructura de directorios resultante de la refactorización a roles.

```

1 ansible
2 |-- ansible.cfg
3 |-- inventory.ini
4 |-- site.yml
5 |-- group_vars
6 |   |-- all.yml
7 |-- roles
8     |-- docker
9     |   |-- defaults
10    |   |   |-- main.yml
11    |   |-- tasks
12    |       |-- main.yml
13    |-- firewall
14    |   |-- tasks
15    |       |-- main.yml
16    |-- grafana_alloy
17    |   |-- defaults
18    |   |   |-- main.yml
19    |   |-- tasks
20    |   |   |-- main.yml
21    |   |-- templates
22    |       |-- config.alloy.j2
23    |-- lgtm_stack
24    |   |-- defaults
25    |   |   |-- main.yml
26    |   |-- tasks
27    |   |   |-- main.yml
28    |   |-- templates
29    |       |-- dashboards-provider.yaml.j2
30    |       |-- datasources.yaml.j2

```

```

31 |         |-- docker-compose.yml.j2
32 |         |-- loki.yml.j2
33 |         |-- mimir.yml.j2
34 |         `-- tempo.yml.j2
35 |     `-- minio
36 |         |-- defaults
37 |         |   |-- main.yml
38 |         |-- tasks
39 |         |   |-- main.yml
40 |         `-- templates
41 |             |-- minio-env.j2

```

Listado 6.3: Estructura de directorios del código Ansible basado en roles

Todo el conjunto se orquesta desde un único fichero, `site.yml`, que sustituye a la ejecución manual y secuencial de los cuatro *playbooks* originales. El listado 6.4 muestra las dos formas de invocarlo: la primera línea despliega el sistema completo, y la segunda restringe la ejecución a una parte concreta.

```

1 ansible-playbook site.yml                # despliegue completo
2 ansible-playbook site.yml --tags minio    # solo una fase concreta

```

Listado 6.4: Órdenes de despliegue completo y parcial sobre `site.yml`

Esa segunda forma se apoya en las etiquetas (*tags*) de Ansible, que permiten marcar cada *play* con uno o varios nombres y, en la invocación, seleccionar cuáles se ejecutan. Cada *play* de `site.yml` lleva la etiqueta del componente que despliega (`docker`, `minio`, `grafana_alloy`, `lgtm_stack`), de modo que `--tags minio` ejecuta únicamente el despliegue del clúster de almacenamiento y omite el resto sin necesidad de mantener ficheros separados para cada fase.

El listado 6.5 muestra el contenido de `site.yml`, que asocia cada grupo de hosts del inventario con los roles que le corresponden.

```

1 - name: Instalar Docker en todos los hosts
2   hosts: all
3   become: true
4   tags: [docker]
5   roles:
6     - docker
7
8 - name: Desplegar MinIO Open Source distribuido
9   hosts: minio
10  become: true
11  tags: [minio]

```

```
12  roles:
13    - firewall
14    - minio
15
16  - name: Desplegar Grafana Alloy
17    hosts: monitoring
18    become: true
19    tags: [grafana_alloy]
20    roles:
21      - firewall
22      - grafana_alloy
23
24  - name: Desplegar stack LGTM
25    hosts: minio-1
26    become: true
27    tags: [lgtm_stack]
28    roles:
29      - firewall
30      - lgtm_stack
```

Listado 6.5: Extracto de `site.yml`: asociación de grupos de hosts y roles

El resultado de esta organización es el que se perseguía con el objetivo de la refactorización: el despliegue completo del sistema, sobre las cuatro máquinas y con sus cinco componentes, se reduce a la ejecución de un único comando, el de la primera línea del listado 6.4, sin ningún paso manual intermedio ni ningún orden que el operador tenga que recordar, ya que es el propio `site.yml` el que fija la secuencia. Y, gracias a las etiquetas, esa misma definición sirve además para desplegar solo una parte del sistema cuando se está trabajando sobre ella, sin necesidad de mantener una versión reducida del despliegue en paralelo.

6.2 Implementación de almacenamiento y observabilidad

Esta fase despliega, sobre la infraestructura base ya aprovisionada en la fase previa, los tres roles que materializan el almacenamiento distribuido y la observabilidad completa del sistema.

6.2.1 Rol `minio`

Este rol instala el paquete de MinIO (`.deb` o `.rpm` según el sistema operativo detectado mediante la variable interna `ansible_os_family`) y deja el nodo listo para formar parte del clúster distribuido. El listado 6.6 recoge un extracto representativo de sus tareas, sobre el que puede seguirse cada uno de los pasos que se describen a continuación.

El servicio no se ejecuta como `root`: las líneas 6-19 crean el grupo y el usuario de sistema `minio-user`, sin directorio personal y sin intérprete de comandos interactivo, que es el usuario bajo el que arranca la unidad de `systemd`. Las líneas 21-34 preparan el disco adicional que el `Vagrantfile` añadió al nodo, formateándolo como `ext4` y montándolo en `/mnt/disk1`; ambas tareas se condicionan a que el disco se haya detectado realmente, para no actuar sobre una máquina que no lo tenga. La configuración del modo distribuido se concentra en la variable `minio_volumes` (línea 3), que emplea la notación de expansión de MinIO (`{1..2}`) para declarar los dos nodos del clúster en una sola línea, y que se materializa en el fichero de entorno del servicio mediante la plantilla de las líneas 45-55. Para que esos nombres se resuelvan, las líneas 36-43 escriben las entradas correspondientes en `/etc/hosts` de cada nodo. Es el único componente del sistema que no se despliega en contenedor sino como servicio nativo de `systemd`, por las razones expuestas en la sección 5.2.3. Genera además un *token* de autenticación Prometheus del propio clúster MinIO (mediante `mc admin prometheus generate`) y lo persiste en un fichero del nodo `minio-1`, de forma que el rol `grafana_alloy` pueda consumirlo posteriormente para el *scraping* de las métricas nativas del clúster. Se guarda en un fichero, y no solo en una variable en memoria, para que siga disponible incluso desde una ejecución independiente de `ansible-playbook`, en la que un dato registrado solo en memoria no sobreviviría. Tanto la generación del *token* como su escritura se condicionan a que el fichero no exista todavía, de forma que el *token* se genera una sola vez y las ejecuciones posteriores no lo alteran; el motivo de esa condición, que no es evidente a primera vista, se explica en la sección 7.4.

```
1 # roles/minio/defaults/main.yml
2 # MinIO usa notacion de expansion {x..y} para nodos secuenciales
3 minio_volumes: "http://minio-{1..2}.local:9000/mnt/disk1/minio"
4
5 # roles/minio/tasks/main.yml
6 - name: Crear grupo minio-user
7   group:
8     name: "{{ minio_group }}"
9     system: yes
10    state: present
11
12 - name: Crear usuario minio-user (sin home directory)
13   user:
14     name: "{{ minio_user }}"
15     group: "{{ minio_group }}"
16     system: yes
17     create_home: no
18     shell: /usr/sbin/nologin
```

```

19     state: present
20
21 - name: Formatear particion del disco adicional como ext4
22   filesystem:
23     fstype: ext4
24     dev: "{{ minio_disk.stdout }}"
25   when: minio_disk.stdout != ""
26
27 - name: Montar particion en /mnt/disk1
28   mount:
29     path: /mnt/disk1
30     src: "{{ minio_disk.stdout }}"
31     fstype: ext4
32     opts: defaults
33     state: mounted
34   when: minio_disk.stdout != ""
35
36 - name: Anadir entradas a /etc/hosts para cluster MinIO
37   blockinfile:
38     path: /etc/hosts
39     block: |
40         192.168.56.10 minio-1.local minio-1
41         192.168.56.11 minio-2.local minio-2
42     marker: "# {mark} ANSIBLE MANAGED BLOCK - MinIO Cluster"
43     state: present
44
45 - name: Crear archivo de configuracion /etc/default/minio
46   template:
47     src: minio-env.j2
48     dest: "{{ minio_config_file }}"
49     owner: "{{ minio_user }}"
50     group: "{{ minio_group }}"
51     mode: '0640'
52
53 # roles/minio/templates/minio-env.j2 (extracto)
54 MINIO_VOLUMES="{{ minio_volumes }}"
55 MINIO_OPTS="--console-address :{{ minio_console_port }}"

```

Listado 6.6: Extracto del rol minio: usuario de servicio, preparación del disco y configuración del modo distribuido

6.2.2 Rol grafana_alloy

Despliega el agente Grafana Alloy, en un contenedor Docker configurado con `network_mode: host`, en los nodos `app-monitor-*`. Su configuración se genera a partir de una plantilla Jinja2 (`config.alloy.j2`) que activa o desactiva bloques enteros de *pipeline* en función del inventario. El listado 6.7 recoge uno de esos bloques condicionales: el *scraping* de métricas de la aplicación de demostración TNS queda envuelto en el condicional de la línea 1, de modo que solo se incluye en los nodos listados en la variable `tns_hosts`. Nótese que el destino al que se reenvían esas métricas (líneas 17-21) sí es incondicional, porque todos los nodos envían a Mimir sus métricas de sistema aunque no ejecuten TNS.

```

1 {% if inventory_hostname in tns_hosts %}
2 // Metricas de la app TNS: scrape de los servicios db/app/loadgen
3 prometheus.scrape "tns_apps" {
4   targets = [
5     { "__address__" = "localhost:8000", "job" = "tns/db",
6       "instance" = "{{ inventory_hostname }}", "namespace" = "tns" },
7     { "__address__" = "localhost:8001", "job" = "tns/app",
8       "instance" = "{{ inventory_hostname }}", "namespace" = "tns" },
9     { "__address__" = "localhost:8002", "job" = "tns/loadgen",
10      "instance" = "{{ inventory_hostname }}", "namespace" = "tns" },
11   ]
12   scrape_interval = "15s"
13   forward_to = [prometheus.remote_write.mimir.receiver]
14 }
15 {% endif %}
16
17 prometheus.remote_write "mimir" {
18   endpoint {
19     url = "{{ mimir_url }}"
20   }
21 }

```

Listado 6.7: Extracto de la plantilla `config.alloy.j2`: bloque condicional según el inventario

Las dos tareas que llevan esa plantilla hasta un contenedor en marcha son las del listado 6.8. La primera (líneas 2-7) renderiza `config.alloy` en el nodo y, gracias a `register`, deja disponible el *checksum* del fichero resultante; la segunda (líneas 9-30) declara el contenedor, con el `network_mode: host` ya mencionado (línea 15) y con los montajes que Alloy necesita para leer los logs del sistema y hablar con el demonio de Docker (líneas 20-24).

```

1 # roles/grafana_alloy/tasks/main.yml
2 - name: Crear configuracion de Alloy

```

```

3  template:
4      src: config.alloy.j2
5      dest: "{{ alloy_config_dir }}/config.alloy"
6      mode: '0644'
7  register: alloy_config
8
9  - name: Ejecutar contenedor Alloy
10     docker_container:
11         name: "{{ alloy_container_name }}"
12         image: "{{ alloy_image }}"
13         state: started
14         restart_policy: always
15         network_mode: host
16
17     labels:
18         config-hash: "{{ alloy_config.checksum }}"
19
20     volumes:
21         - "{{ alloy_config_dir
22           }}/config.alloy:/etc/alloy/config.alloy:ro"
23         - "{{ alloy_data_dir }}:/var/lib/alloy/data"
24         - "/var/log:/var/log:ro"
25         - "/var/run/docker.sock:/var/run/docker.sock"
26
27     command: >
28         run
29         --server.http.listen-addr=0.0.0.0:12345
30         --storage.path=/var/lib/alloy/data
31         /etc/alloy/config.alloy

```

Listado 6.8: Extracto del rol `grafana_alloy`: renderizado de la configuración y despliegue del contenedor

Al igual que el rol `lgtm_stack`, el despliegue es idempotente: el contenedor no se elimina y recrea en cada ejecución, sino que se declara con el módulo `docker_container`, que respeta un contenedor ya en ejecución mientras sus parámetros no cambien. Como el contenido del volumen de configuración montado no forma parte de esa comparación, se añade al contenedor una etiqueta (*label*) `config-hash` con el *checksum* del fichero `config.alloy` renderizado, que es justamente el que la tarea anterior registró (líneas 17-18): al cambiar la configuración cambia el *hash*, la comparación de etiquetas detecta la diferencia y el módulo recrea únicamente ese contenedor para que Alloy relea la configuración; una ejecución sin cambios no lo toca. El rol `lgtm_stack` recurre a esta misma estrategia de etiqueta con *hash*

de configuración (véase la sección siguiente), allí mediante una etiqueta de Docker Compose y aquí mediante una de `docker_container`.

6.2.3 Rol `lgdm_stack`

Despliega los cuatro componentes del stack de observabilidad (Loki, Grafana, Tempo y Mimir) en `minio-1` mediante un fichero `docker-compose.yml` generado desde una plantilla Jinja2 y la orden `docker compose up`, además de crear los *buckets* de MinIO que cada uno necesita como *backend* de almacenamiento S3. El uso de Docker Compose en este rol, en lugar de los módulos `docker_*` de Ansible que sí emplea el rol `grafana_alloy`, tiene una ventaja añadida: la orden `docker compose` no depende del [Software Development Kit \(SDK\)](#) de Python de Docker en el host de destino, por lo que este rol queda al margen del problema de compatibilidad con la librería `requests` descrito en la sección 6.3. El listado 6.9 recoge las cuatro tareas centrales del rol, que resumen esa secuencia: registrar el clúster de MinIO como destino del cliente `mc` (líneas 2-6), crear en él los cinco *buckets* que los servicios del stack necesitan (líneas 8-17), renderizar el fichero de composición a partir de la plantilla Jinja2 (líneas 19-23) y, por último, levantarlo con `docker compose up` (líneas 25-33).

```

1 # roles/lgdm_stack/tasks/main.yml
2 - name: Configurar alias MinIO
3   shell: |
4     mc alias set local http://{{ minio_endpoint }} \
5       {{ minio_access_key }} {{ minio_secret_key }} --insecure
6   changed_when: false
7
8 - name: Crear buckets LGTM
9   shell: |
10    mc mb local/{{ item }} --ignore-existing --insecure
11  loop:
12    - "{{ loki_bucket }}"
13    - "{{ tempo_bucket }}"
14    - "{{ mimir_blocks_bucket }}"
15    - "{{ mimir_ruler_bucket }}"
16    - "{{ mimir_alertmanager_bucket }}"
17  changed_when: false
18
19 - name: Generar docker-compose.yml del stack LGTM
20   template:
21     src: docker-compose.yml.j2
22     dest: "{{ lgdm_config_dir }}/docker-compose.yml"
23     mode: '0644'
24

```

```

25 - name: Levantar stack LGTM con docker compose
26   command: docker compose up -d --remove-orphans
27   args:
28     chdir: "{{ lgtm_config_dir }}"
29   register: compose_up
30   changed_when: >-
31     'Started' in compose_up.stderr
32     or 'Created' in compose_up.stderr
33     or 'Recreated' in compose_up.stderr

```

Listado 6.9: Extracto del rol `lgtm_stack`: creación de los *buckets* y despliegue del stack con Docker Compose

La plantilla que renderiza la tercera de esas tareas es el listado 6.10, del que conviene destacar tres rasgos. El primero es la red común declarada en las líneas 3-5, a la que se conectan los cuatro servicios y que permite que Grafana se dirija a Loki, Tempo y Mimir por su nombre de servicio (`http://loki:3100`) en lugar de por una dirección IP. El segundo es el patrón que siguen los cuatro contenedores, visible en el servicio `loki` de las líneas 9-26: una imagen, un puerto publicado hacia el exterior (línea 16), su fichero de configuración montado desde el anfitrión (línea 18) y la orden de arranque que lo consume (línea 19). El tercero es la etiqueta `config-hash` (línea 26), que es la pieza que hace idempotente el despliegue y cuyo motivo explica el comentario que la precede.

```

1 name: lgtm
2
3 networks:
4   lgtm-network:
5     name: lgtm-network
6
7 services:
8
9   loki:
10    image: grafana/loki:latest
11    container_name: loki
12    restart: always
13    networks:
14      - lgtm-network
15    ports:
16      - "{{ loki_port }}:3100"
17    volumes:
18      - "{{ lgtm_config_dir
19        }}/loki/loki.yaml:/etc/loki/local-config.yaml"
20    command: ["-config.file=/etc/loki/local-config.yaml"]

```

```

20 labels:
21     # Checksum de la config bind-monteada: el contenido de un
22     # bind mount no entra en el hash que calcula "docker compose
23     # up", así que sin esta etiqueta un cambio en loki.yaml no
24     # recrearía nada. Al cambiar la config cambia la etiqueta ->
25     # cambia el hash -> compose recrea SOLO este contenedor.
26     config-hash: "{{ lookup('template', 'loki.yaml.j2') |
27     hash('sha256') }}"
28
29 # ... tempo y mimir, analogos ...
30
31 grafana:
32     image: grafana/grafana:latest
33     container_name: grafana
34     restart: always
35     networks:
36         - lgtm-network
37     ports:
38         - "{{ grafana_port }}:3000"
39     volumes:
40         # ... montajes de datasources.yaml y dashboards-provider.yaml
41         #     bajo /etc/grafana/provisioning/ ...
42         - "{{ lgtm_config_dir
43         }}/grafana/dashboards:/var/lib/grafana/dashboards"
44         - "{{ lgtm_data_dir }}/grafana:/var/lib/grafana"
45     environment:
46         GF_SECURITY_ADMIN_PASSWORD: "admin"
47         GF_USERS_ALLOW_SIGN_UP: "false"
48     labels:
49         config-hash: "{{ (lookup('template', 'datasources.yaml.j2')
50         + lookup('template', 'dashboards-provider.yaml.j2'))
51         | hash('sha256') }}"

```

Listado 6.10: Extracto de la plantilla del `docker-compose.yml` del stack LGTM

Adicionalmente, este rol aprovisiona de forma automática en Grafana los paneles (*dashboards*) de monitorización del sistema y del propio clúster MinIO. El primero se descarga desde grafana.com; el segundo, directamente desde el repositorio oficial de MinIO en GitHub, al haber sido retirado del portal de Grafana. En ambos se sustituye la referencia a la fuente de datos por la configurada en el propio despliegue (Mimir).

6.2.4 Integración con MinIO como *backend* S3

Uno de los objetivos de diseño (sección 5.3) es que los tres componentes de almacenamiento del stack (Loki, Tempo y Mimir) no persistan en disco local, sino sobre el propio clúster de MinIO a través de su API compatible con S3. Cada uno se configura con un bloque de almacenamiento que apunta al extremo de MinIO (192.168.56.10:9000) y al *bucket* que le corresponde. Esos *buckets* no se crean a mano: los crea el propio rol `lgdm_stack` antes de levantar los contenedores, mediante el cliente `mc` y la tarea recogida en las líneas 8-17 del listado 6.9, que recorre en un bucle los cinco nombres necesarios (`loki-data`, `tempo-data` y, para Mimir, `mimir-blocks`, `mimir-ruler` y `mimir-alertmanager`). Solo después, cada servicio recibe la configuración de almacenamiento que apunta al suyo; el listado 6.11 recoge un extracto representativo de los tres ficheros.

```
1 # loki.yaml -> common.storage.s3
2 s3:
3   endpoint: 192.168.56.10:9000
4   bucketnames: loki-data
5   access_key_id: minioadmin
6   secret_access_key: minioadmin123
7   region: us-east-1
8   s3forcepathstyle: true
9   insecure: true
10 # mimir.yaml -> blocks_storage.s3 (ruler y alertmanager, analogos)
11 s3:
12   endpoint: 192.168.56.10:9000
13   bucket_name: mimir-blocks
14   region: us-east-1
15   insecure: true
16 # tempo.yaml -> storage.trace.s3
17 s3:
18   endpoint: 192.168.56.10:9000
19   bucket: tempo-data
20   region: us-east-1
21   forcepathstyle: true
22   insecure: true
```

Listado 6.11: Extracto de la configuración de almacenamiento S3 de Loki, Mimir y Tempo hacia MinIO

Tres detalles de esta integración merecen mención, pues son consecuencia directa de usar MinIO, y no Amazon S3, como *backend*, y su ausencia fue causa de fallos reales durante el desarrollo:

- **Direccionamiento *path-style*** (`s3forcepathstyle` en Loki, `forcepathstyle` en Tempo, ambos a `true`). Los clientes S3 emplean por defecto el direccionamiento *virtual-hosted* de AWS, en el que el nombre del *bucket* se antepone al del host (`http://bucket.s3...`). MinIO, servido desde una dirección IP fija, requiere el direccionamiento *path-style*, en el que el *bucket* va en la ruta (`http://192.168.56.10:9000/bucket`); sin este ajuste, el cliente intentaría resolver un nombre de host inexistente. Mimir es la excepción, y por eso su configuración no incluye ninguna opción equivalente: su cliente S3 resuelve el modo de direccionamiento de forma automática y escoge *path-style* para cualquier extremo que no corresponda a uno de los proveedores de nube conocidos, de modo que el ajuste explícito solo hace falta en los otros dos.
- **Región obligatoria** (`region: us-east-1`). Aunque MinIO ignora la región de AWS, el cliente S3 de Mimir la exige como parámetro no vacío; se fija a un valor convencional en los cuatro bloques de almacenamiento de Mimir (común, bloques, ruler y alertmanager), además de en Loki y Tempo.
- **Conexión sin TLS** (`insecure: true`). Coherente con la decisión de alcance descrita en la sección 5.6, la comunicación con MinIO es en texto claro; en un despliegue de producción, este indicador se retiraría junto con la habilitación de TLS en MinIO.

Con estos tres ajustes, los datos de telemetría del stack pasan a residir en el mismo clúster que el TFG estudia, según el criterio de diseño planteado en la sección 5.3.

6.2.5 Adaptación de la aplicación de demostración TNS

La carga de trabajo que ejercita todo el *pipeline* es [TNS](#) [22], una aplicación de ejemplo publicada por Grafana Labs y pensada precisamente como banco de pruebas de observabilidad. Su distribución original no es solo la aplicación: el `docker-compose.yml` del repositorio oficial levanta, junto a los servicios de la aplicación propiamente dicha, una plataforma de observabilidad completa en miniatura (Grafana junto con sus *backends* de métricas, logs y trazas, y el agente de recolección que los alimenta), de manera que quien clona el repositorio obtiene un entorno autocontenido y listo para explorar sin depender de nada externo.

Ese es justamente el motivo por el que aquí no se emplea tal cual. En este proyecto la plataforma de observabilidad ya existe, es el stack LGTM desplegado por el rol `lgtm_stack` sobre `minio-1`, y es precisamente el sistema que se quiere validar. Reutilizar la que trae TNS habría levantado una segunda plataforma paralela en cada nodo `app-monitor-*`, duplicando servicios, compitiendo por unos recursos ya ajustados (1 GB de RAM por nodo, tabla 6.1) y, sobre todo, vaciando de sentido la prueba: la telemetría acabaría en un Grafana local en lugar de recorrer el *pipeline* real bajo estudio.

El fichero `docker-compose.yml` de este repositorio es, por tanto, una versión reducida que conserva únicamente los tres servicios que constituyen la aplicación en sí y que son los que generan datos de telemetría real:

- `db`, el servicio de datos, expuesto en el puerto 8000.
- `app`, la aplicación web, que consulta a `db` y se expone en el puerto 8001.
- `loadgen`, el generador de carga, que ejerce peticiones continuas contra `app` y se expone en el puerto 8002. Es la pieza que hace que el entorno produzca métricas, logs y trazas de forma sostenida sin intervención manual.

A esa reducción se añaden dos ajustes en la definición de los tres servicios, ambos visibles en el listado 6.12, que recoge el fichero resultante. El primero es redirigir el destino de las trazas: la variable `JAEGER_ENDPOINT` (línea 8) apunta al receptor Jaeger nativo que Tempo expone en `minio-1` (`http://192.168.56.10:14268/api/traces`) en lugar del *backend* de trazas local que traía la composición original, decisión de diseño justificada en la sección 5.3.1. El segundo es la variable `JAEGER_TRACEID_128BIT` (línea 13), que corrige el desajuste de formato de identificadores descrito en la sección 6.2.6. Cada servicio conserva además la etiqueta `namespace: tns` (líneas 14-15), que es la que permite a Grafana Alloy identificarlos y a los paneles agrupar su telemetría. La cadena de dependencias entre los tres se declara con `depends_on` (líneas 20 y 32) y reproduce el sentido del tráfico: el generador de carga llama a la aplicación, y esta al servicio de datos.

```

1 services:
2   db:
3     image: grafana/tns-db:47b160f
4     command: ["-log.level=debug"]
5     ports:
6       - "8000:80"
7     environment:
8       JAEGER_ENDPOINT: "http://192.168.56.10:14268/api/traces"
9       JAEGER_TAGS: job=tns/db
10      JAEGER_SAMPLER_TYPE: const
11      JAEGER_SAMPLER_PARAM: 1
12      # Sin esto Jaeger emite IDs de 64 bits y el salto a logs falla
13      JAEGER_TRACEID_128BIT: "true"
14     labels:
15       namespace: tns
16
17   app:
18     image: grafana/tns-app:47b160f

```

```

19  command: ["-log.level=debug", "http://db"]
20  depends_on:
21    - db
22  ports:
23    - "8001:80"
24  environment:
25    # ... identicas a las de db, con JAEGER_TAGS: job=tns/app ...
26  labels:
27    namespace: tns
28
29  loadgen:
30    image: grafana/tns-loadgen:47b160f
31    command: ["-log.level=debug", "http://app"]
32    depends_on:
33      - app
34    ports:
35      - "8002:80"
36    environment:
37      # ... identicas a las de db, con JAEGER_TAGS: job=tns/loadgen ...
38    labels:
39      namespace: tns

```

Listado 6.12: Fichero `docker-compose.yml` de la aplicación TNS empleado en este proyecto

6.2.6 Correlación de trazas y logs en Grafana

El cuarto objetivo del TFG no se limita a que cada señal llegue a su almacén, sino a que las tres puedan correlacionarse entre sí: poder pasar, desde una traza concreta, a los logs generados durante esa misma petición, y viceversa. Esta capacidad no es automática; se configura en los *datasources* que el rol `lgtm_stack` aprovisiona en Grafana.

Los tres *datasources* (Loki, Tempo y Mimir) se declaran con un identificador (`uid`) fijo y conocido de antemano. Este detalle es lo que permite que tanto los paneles aprovisionados automáticamente (que referencian el `uid` `mimir`, subsección anterior) como los enlaces de correlación se resuelvan sin ningún paso manual en la interfaz: cada *datasource* puede referirse a los otros por un `uid` estable en lugar del identificador aleatorio que Grafana asignaría. Sobre esa base, el *datasource* de Tempo incorpora un bloque `tracesToLogsV2` y el de Loki un bloque `derivedFields`, que definen el enlace en cada sentido, como muestra el listado 6.13.

```

1  - name: Loki
2    type: loki
3    uid: loki
4    url: http://loki:3100

```

```

5  jsonData:
6    derivedFields:           # salto log -> traza
7      - name: TraceID
8        matcherType: regex
9        matcherRegex: 'traceID=(\w+)'
10       url: '$${__value.raw}' # "$$" escapa un "$" literal
11       datasourceUid: tempo
12
13 - name: Tempo
14   type: tempo
15   uid: tempo
16   url: http://tempo:3200
17   jsonData:
18     tracesToLogsV2:         # salto traza -> logs
19       datasourceUid: loki
20       spanStartTimeShift: "-1m"
21       spanEndTimeShift: "1m"
22       customQuery: true
23       query: '{host=~".+"} |= "$${__span.traceId}"'
24   serviceMap:
25     datasourceUid: mimir    # grafo de servicios desde metricas de
                               spans

```

Listado 6.13: Correlación bidireccional traza-logs en los *datasources* de Loki y Tempo (*datasources.yaml*)

El mecanismo funciona en las dos direcciones. En el sentido traza → logs, al visualizar una traza en Grafana cada *span* ofrece un salto directo a Loki: la consulta LogQL definida en query (línea 23 del listado 6.13) filtra las líneas que contienen el *traceID* del *span* (`${__span.traceId}`), acotadas además a una ventana temporal de un minuto antes y después del mismo (*spanStartTimeShift*/*spanEndTimeShift*), lo que evita recorrer todo el histórico de logs. Para que ese filtrado devuelva resultados, las líneas de log deben contener el *traceID*: los servicios de la aplicación TNS lo emiten en su salida (`level=warn traceID=0b43d0612539fa88 msg="GET / ..."`), que Grafana Alloy recolecta vía *discovery.docker* y remite a Loki (sección 6.2). En el sentido inverso, logs → traza, el bloque *derivedFields* del *datasource* de Loki extrae ese mismo identificador de la línea de log mediante la expresión regular `traceID=(\w+)` (línea 9 del mismo listado) y lo convierte en un enlace directo a Tempo; el cuantificador `+` exige al menos un carácter, de modo que las peticiones que no generan traza (y que TNS registra como `traceID=` vacío) no producen enlaces rotos.

Selector de flujo de LogQL

Un detalle no evidente es que la configuración anterior es el resultado de corregir un problema que ilustra bien lo poco automática que es esta correlación. La primera versión no definía `query` sino la opción `filterByTraceID: true`, que delega en Grafana la construcción de la consulta. Grafana la construye a partir del campo `tags`, que asocia atributos del `span` con etiquetas de Loki y que, al no declararse, toma un valor por defecto (`cluster`, `namespace`, `pod`, `service.name`...). Ninguna de esas etiquetas existe en esta instalación: las únicas que Alloy adjunta a los logs son `host`, `instance` y `service_name`. El selector de flujo resultante quedaba vacío y Loki rechazaba la consulta con un error de análisis, ya que LogQL exige al menos un *matcher* que no admita el valor vacío. El efecto práctico era que el botón «Logs for this span» fallaba en lugar de devolver resultados, pese a que tanto las trazas como los logs correlacionados estaban correctamente almacenados.

La solución adoptada es construir la consulta de forma explícita con `customQuery`, empleando `{host=~".+"}` como selector: es un *matcher* no vacío válido y, al no fijar un valor concreto, funciona igual desde cualquiera de los nodos `app-monitor-*`. La alternativa habría sido declarar `tags` para mapear atributos del `span` a etiquetas de Loki, pero resulta más frágil aquí: las trazas de TNS llegan por el protocolo Jaeger y sus atributos de proceso no contienen el nombre del nodo (el `hostname` que reportan es el identificador del contenedor, no `app-monitor-1`), por lo que no hay ningún atributo del `span` que se corresponda con la etiqueta `host` de los logs.

Identificadores de traza de 64 y 128 bits

El segundo obstáculo no estaba en la configuración de Grafana sino en un desajuste de formato entre los dos extremos que se pretendía correlacionar. El cliente Jaeger con el que viene instrumentada la aplicación TNS genera, por omisión, identificadores de traza de 64 bits, que escribe en sus registros como 16 dígitos hexadecimales (`traceID=2af97d9198d034fc`). Tempo, en cambio, almacena internamente identificadores de 128 bits, de modo que al recibir una traza por el receptor Jaeger la completa con ceros por la izquierda hasta los 32 dígitos (`00000000000000002af97d9198d034fc`), que es la forma que Grafana muestra en la interfaz y la que sustituye en `${__span.traceId}`.

La consecuencia es que el filtro de línea generado buscaba literalmente la cadena rellenada con ceros, que no aparece en ningún registro, y el salto traza → logs devolvía siempre un resultado vacío pese a que ambos identificadores designaban la misma traza. Es un fallo especialmente engañoso porque la consulta es sintácticamente válida, se ejecuta sin error y el identificador mostrado en pantalla parece, a simple vista, el correcto. La dirección inversa, logs → traza, no se veía afectada: la API de Tempo acepta indistintamente ambas formas, de manera que el enlace construido por `derivedFields` a partir del identificador corto resolvía

correctamente. Este comportamiento asimétrico fue, de hecho, lo que orientó el diagnóstico hacia un problema de formato y no de configuración.

La corrección se aplica en el origen, añadiendo la variable de entorno `JAEGER_TRACEID_128BIT` a los tres servicios de TNS en `docker-compose.yml`, con lo que el cliente Jaeger emite ya identificadores de 128 bits y la cadena que aparece en los registros coincide carácter a carácter con la que almacena Tempo. Se prefirió esta solución a intentar absorber la diferencia en la consulta LogQL, que no dispone de ninguna primitiva para eliminar los ceros iniciales de una variable interpolada.

El escapado del símbolo \$ en el aprovisionamiento

Un tercer detalle, de naturaleza distinta pero con un efecto igualmente silencioso, afecta a las dos direcciones de la correlación. Ambas configuraciones dependen de variables de plantilla de Grafana (`${__span.traceId}` y `${__value.raw}`) que se resuelven en tiempo de consulta, cuando el usuario pulsa el enlace. Sin embargo, Grafana interpola variables de entorno en los ficheros de aprovisionamiento en el momento de cargarlos, y esa interpolación se aplica a cualquier construcción con la forma `${...}`. Como no existe ninguna variable de entorno con esos nombres, Grafana las sustituía por la cadena vacía antes de que llegasen a usarse: el *datasource* quedaba registrado con una consulta `|= ""` y un enlace con `url` vacía. El resultado era una configuración que se cargaba sin ningún error en los registros, y que por tanto parecía correcta, pero en la que el salto traza $\rightarrow$ logs devolvía todos los logs del nodo en lugar de los de la traza, y el salto inverso no llevaba a ninguna parte. La solución es escapar el símbolo duplicándolo (`$$`), que es la convención que Grafana define para representar un \$ literal en estos ficheros. El diagnóstico de este tipo de fallo obliga a inspeccionar la configuración efectivamente cargada por Grafana, consultable a través de su API de administración (`/api/datasources/uid/tempo`), y no únicamente el fichero YAML generado por Ansible: es precisamente entre uno y otro donde se producía la pérdida.

Métricas derivadas de spans: el *metrics-generator*

El bloque `serviceMap` del listado 6.13 habilita en Grafana la pestaña de grafo de servicios, que representa gráficamente qué servicio llama a cuál y con qué latencia y tasa de error. Ese grafo no se dibuja a partir de las trazas directamente, sino de un conjunto de series (`traces_service_graph_*`) que debe existir previamente en Mimir. Quien las produce es el *metrics-generator* de Tempo, un componente que procesa los *spans* entrantes, deriva de ellos métricas agregadas y las envía por `remote_write`. Al no estar habilitado inicialmente, apuntar `serviceMap` a Mimir no bastaba: el *datasource* era correcto pero la pestaña aparecía siempre vacía, porque nadie generaba esas series. Su activación requiere dos piezas complementarias en `tempo.yml`, como recoge el listado 6.14.

```
1 metrics_generator:
2   registry:
3     external_labels:
4       source: tempo
5   storage:
6     path: /tmp/tempo/generator/wal
7   remote_write:
8     - url: http://mimir:9009/api/v1/push
9     send_exemplars: true
10
11 # Los procesadores NO se activan en el bloque anterior, sino via
12 # overrides: por defecto Tempo no ejecuta ninguno, de modo que
13 # definir metrics_generator sin esto no genera ninguna metrica.
14 overrides:
15   defaults:
16     metrics_generator:
17       processors: [service-graphs, span-metrics]
```

Listado 6.14: Activación del *metrics-generator* de Tempo (*tempo.yaml*)

La separación entre ambas piezas es la principal fuente de confusión: el bloque `metrics_generator` configura cómo se escriben las métricas (destino y etiquetas), mientras que la lista de procesadores que determina qué se genera vive en el bloque `overrides`, y su valor por defecto es vacío. Configurar solo lo primero produce un Tempo que arranca sin errores y no genera absolutamente nada. Los dos procesadores activados (línea 17 del listado 6.14) cubren los dos usos previstos: `service-graphs` produce las series `traces_service_graph_*` que alimentan el grafo de servicios, y `span-metrics` las series `traces_spanmetrics_*` con métricas RED (*rate*, *errors*, *duration*) por *span*. El envío a Mimir no necesita cabecera de tenant porque Mimir se despliega con `multitenancy_enabled: false`, de modo que todas las series van al tenant anónimo.

Sobre este flujo de diagnóstico, de una traza de TNS en Tempo a sus logs correlacionados en Loki mediante un mismo `traceID`, se apoya la validación de extremo a extremo del capítulo 7 (sección 7.1).

6.3 Problemas de compatibilidad multi-sistema operativo

Dar soporte a las tres distribuciones (Debian, Rocky Linux y AlmaLinux) sacó a la luz varios problemas de compatibilidad no triviales, cuya identificación, diagnóstico y solución constituyen una parte sustancial del trabajo realizado. Las tres subsecciones siguientes describen los principales, en cada caso con el síntoma observado, la causa real una vez diagnosticada

y la solución finalmente incorporada a los roles.

6.3.1 **firewalld** y las tablas persistentes de **nftables**

En Rocky Linux y AlmaLinux, `firewalld` [26] viene activo de fábrica y bloquea los puertos necesarios para el funcionamiento distribuido del clúster (9000/9001 de MinIO, 12345 de Alloy, y los puertos OTLP/Jaeger del stack LGTM). Detener el servicio de forma directa no es suficiente: la tabla de `nftables` [27] que crea `firewalld` se marca internamente con los indicadores `owner`, `persist`, lo que significa que solo el proceso que la creó puede eliminarla, y que la tabla sobrevive aunque ese proceso termine. Si el servicio se detiene sin haber sido él mismo quien reclama esa propiedad (por ejemplo, si arrancó en un ciclo de arranque anterior al aprovisionamiento con Ansible), la tabla queda huérfana pero activa, rechazando silenciosamente el tráfico TCP aunque `systemctl status` informe del servicio como inactivo.

Este comportamiento provocó, en las primeras pruebas sobre Rocky Linux, una pérdida de **quórum** en el clúster de MinIO: el ping entre `minio-1` y `minio-2` funcionaba con normalidad (el tráfico ICMP no estaba bloqueado), pero las conexiones TCP al puerto 9000 se rechazaban, provocando errores de tipo `SlowDownWrite` en Loki, Tempo y Mimir al intentar escribir en el *backend* S3. La solución robusta consiste en arrancar `firewalld` en primer lugar, para que reclame la propiedad de su tabla persistente, y detenerlo únicamente después, dejando que sea el propio servicio quien la elimine de forma correcta. El listado 6.15 recoge las tres tareas en que se traduce esa maniobra en el rol `firewall`. Conviene fijarse en que ninguna de las dos acciones es incondicional: la primera tarea calcula un hecho (`firewalld_needs_neutralizing`) que resume si queda algo por neutralizar, y las otras dos se supeditan a él mediante `when`. Esa condición es lo que evita que cada nueva ejecución vuelva a arrancar y parar un servicio ya neutralizado, un punto que se analiza en detalle al medir la idempotencia del despliegue (sección 7.4).

```

1 # La maniobra solo se ejecuta si queda algo que neutralizar: el
2 # servicio activo, habilitado para el proximo arranque, o una tabla
3 # nftables suya huérfana en el kernel. En caso contrario el estado
4 # deseado ya se cumple y no se toca nada (ver idempotencia).
5 - name: Determinar si hace falta neutralizar firewalld
6   set_fact:
7     firewalld_needs_neutralizing: >-
8     {{ firewalld_present
9        and (firewalld_active.stdout | default('') | trim == 'active'
10           or firewalld_enabled.stdout | default('') | trim ==
11              'enabled'
           or 'firewalld' in (firewalld_nft_tables.stdout |

```

```

    default('')))) }}
12
13 - name: Arrancar firewalld (para que reclame su tabla nftables
    persistente)
14   systemd:
15     name: firewalld
16     state: started
17   when: firewalld_needs_neutralizing
18
19 - name: Detener y deshabilitar firewalld
20   systemd:
21     name: firewalld
22     state: stopped
23     enabled: no
24   when: firewalld_needs_neutralizing

```

Listado 6.15: Neutralización correcta de firewalld (rol firewall)

6.3.2 SDK de Python para Docker y el esquema http+docker

Los módulos de Ansible que gestionan contenedores Docker [15] (`docker_image`, `docker_container`, `docker_network`) se ejecutan en el host de destino, no en el nodo controlador. Rocky Linux y AlmaLinux no traen las librerías Python `requests` y `docker` que necesita este SDK, y la versión de la colección `community.docker` que acompaña al paquete `ansible` de estas distribuciones es antigua: no incluye el *workaround* necesario para funcionar con `requests >= 2.32`, versión en la que cambia el registro del esquema `http+docker` y aparece el error `Not supported URL scheme http+docker`, ya documentado por la comunidad del proyecto [28]. La solución adoptada es fijar una versión de `requests` anterior a la ruptura, más robusta que actualizar la colección, cuya resolución de rutas fija aquí el propio paquete RPM. El listado 6.16 muestra la tarea: se instala con `pip`, condicionada a `ansible_os_family == "RedHat"`, ya que Debian no está afectado y trae ambas librerías en sus paquetes.

```

1 - name: Instalar Docker SDK para Python y requests (Rocky/Alma)
2   pip:
3     name:
4       - "requests==2.31.0"
5       - docker
6     executable: pip3
7   when: ansible_os_family == "RedHat"

```

Listado 6.16: Fijado de versión de requests en el rol docker

6.3.3 `secure_path` de `sudo` y binarios en `/usr/local/bin`

Cuando se invoca un comando por su nombre a secas (`minio`, en lugar de su ruta completa `/usr/local/bin/minio`), el intérprete de comandos no sabe de antemano dónde está ese programa: recorre en orden una lista de directorios buscando un ejecutable con ese nombre, y se queda con la primera coincidencia. Esa lista es la variable de entorno `PATH`. Si el binario está en un directorio que no figura en el `PATH`, la búsqueda termina sin encontrarlo y el intérprete responde `orden no encontrada`, aunque el fichero exista y sea perfectamente ejecutable: «no encontrada» significa aquí «no encontrada en los directorios en los que busca por defecto».

La particularidad de `sudo` (y, por tanto, de cualquier tarea de Ansible con `become: true`, que se apoya en él) es que no emplea el `PATH` del usuario, sino uno propio, fijo y controlado por el administrador, definido en `/etc/sudoers` bajo el nombre de `secure_path`. Es una medida de seguridad deliberada: si `sudo` respetara el `PATH` del usuario, bastaría con colocar un programa malicioso llamado, por ejemplo, `ls` en un directorio situado al principio de esa lista para que se ejecutara como `root` en cuanto la víctima lanzara `sudo ls`. Fijar un `secure_path` conocido cierra esa vía.

El problema es que ese `secure_path` por defecto difiere entre familias de distribuciones: Debian incluye `/usr/local/bin` en él, mientras que Rocky Linux y AlmaLinux lo dejan fuera. La consecuencia práctica es un fallo asimétrico según quién invoque el binario. Un ejecutable instalado en `/usr/local/bin` funciona al llamarlo por su nombre desde una sesión interactiva normal (cuyo `PATH` sí incluye esa ruta), pero falla con `orden no encontrada` al invocarlo desde una tarea con `become: true` sobre Rocky o Alma, porque el `secure_path` que `sudo` impone en su lugar no la contiene. El mismo fichero, con los mismos permisos, se ejecuta o no según el contexto desde el que se le llame; de ahí que el diagnóstico obvio, entrar por SSH y comprobar a mano que el binario está donde debe, no lleve a ninguna parte: da resultado satisfactorio mientras la tarea sigue fallando.

El problema afecta a los dos binarios que este proyecto instala fuera de la gestión de paquetes del sistema, y a ambos por el mismo motivo: `/usr/local/bin` es precisamente la ruta convencional para el software instalado al margen del gestor de paquetes.

- El binario del **servidor MinIO**. Tanto el paquete `.rpm` (Rocky/Alma) como el `.deb` (Debian) lo instalan en `/usr/local/bin/minio`, y no en `/usr/bin/minio`. La solución aplicada es un enlace simbólico hacia `/usr/bin/minio`, que resuelve a la vez dos problemas distintos: por un lado, la tarea de verificación de la instalación invoca el binario por su ruta absoluta `/usr/bin/minio` y no lo encontraría sin el enlace; por otro, cualquier invocación de `minio` desde una tarea con `become: true` quedaría oculta por el `secure_path` que se acaba de describir. Conviene no confundir ambas

causas: que el paquete instale en `/usr/local/bin` es solo el motivo de que el binario esté en una ruta incómoda; el `secure_path` es el motivo de que esa ruta, además, falle bajo `sudo` en la familia RedHat. El enlace se crea de forma incondicional, sin condicionarlo a `ansible_os_family`, porque la ruta de instalación es la misma en las tres distribuciones y no tiene sentido tratarlo como un caso especial de la familia RedHat.

- El **cliente de línea de comandos mc** (MinIO Client), que el rol descarga directamente del sitio oficial en lugar de instalarlo desde un repositorio. Aquí no hace falta enlace alguno: basta con descargarlo ya sobre `/usr/bin/mc`, que es la política que se sigue.

El listado 6.17 recoge las dos tareas correspondientes, que resuelven el mismo problema por caminos distintos: la primera (líneas 2-6) crea el enlace simbólico que pone el servidor de MinIO al alcance de `/usr/bin`, mientras que la segunda (líneas 8-13) evita el problema de raíz descargando `mc` directamente sobre `/usr/bin/mc`, sin pasar por `/usr/local/bin`.

```
1 # roles/minio/tasks/main.yml
2 - name: Crear symlink de minio en /usr/bin
3   file:
4     src: /usr/local/bin/minio
5     dest: /usr/bin/minio
6     state: link
7
8 - name: Instalar MinIO Client (mc)
9   get_url:
10    url: https://dl.min.io/client/mc/release/linux-amd64/mc
11    dest: /usr/bin/mc
12    mode: '0755'
13    run_once: true
```

Listado 6.17: Tareas del rol `minio` que dejan ambos binarios accesibles bajo `secure_path`

De aquí se deriva un criterio general para cualquier ampliación futura del proyecto: todo binario que se instale a mano en `/usr/local/bin` y vaya a invocarse desde una tarea con `become: true` debe quedar accesible desde `/usr/bin`, bien descargándolo directamente allí, bien mediante un enlace simbólico.

6.4 Herramienta de validación automatizada

Para facilitar la reproducibilidad de las pruebas descritas en el capítulo 7 sobre los tres sistemas operativos soportados, se ha desarrollado un script auxiliar, `scripts/test-provision.sh`, que automatiza el ciclo completo de una prueba: selección del

sistema operativo en `config.yml`, despliegue de las cuatro máquinas y ejecución del aprovisionamiento basado en roles (`site.yml`). Sobre ese ciclo básico, tres modos opcionales cubren el resto de las comprobaciones: el despliegue de la aplicación de demostración TNS en ambos nodos `app-monitor-*` junto con la comprobación de que Mimir, Loki y Tempo están recibiendo datos y de que la telemetría llega efectivamente a los *buckets* de MinIO; la medición de *idempotencia*, que ejecuta `site.yml` una segunda vez y verifica que ninguna máquina registra cambios (sección 7.4); y la destrucción del entorno al terminar, que no se aplica por defecto para permitir inspeccionarlo. Cada ejecución queda registrada en un fichero de log independiente para su posterior consulta.

El listado 6.18 recoge las comprobaciones que ejecuta el modo de validación de telemetría, que son las que respaldan buena parte de la matriz de resultados del capítulo 7. Tras desplegar TNS en los dos nodos de aplicación, el script espera un minuto (líneas 5-6) para que Alloy complete al menos un ciclo de recolección, y a continuación interroga a los tres almacenes por separado: a Mimir por las series que tiene en memoria y por los valores distintos de la etiqueta `job`, cuya presencia de entradas `tns/*` demuestra que las métricas de la aplicación llegan (líneas 8-11); a Loki por las etiquetas que ha ingerido (línea 13); y a Tempo por sus etiquetas de búsqueda, que solo existen si ha recibido trazas (línea 15). La segunda mitad del listado cubre la persistencia en el almacenamiento: como Loki y Mimir mantienen los datos recientes en memoria, el script fuerza un vaciado inmediato (líneas 19-20) y después lista el contenido de los tres *buckets* de MinIO (líneas 23-27), de modo que la evidencia no es que los servicios respondan, sino que los objetos están efectivamente escritos.

```
1 run_tns_validation() {
2   vagrant ssh app-monitor-1 -c "cd /vagrant && docker compose up -d"
3   vagrant ssh app-monitor-2 -c "cd /vagrant && docker compose up -d"
4
5   # Alloy necesita al menos un ciclo de scrape completo
6   sleep 60
7
8   vagrant ssh minio-1 -c "curl -s 'http://192.168.56.10:9009\
9 /prometheus/api/v1/query?query=up'"
10  vagrant ssh minio-1 -c "curl -s 'http://192.168.56.10:9009\
11 /prometheus/api/v1/label/job/values'"
12
13  vagrant ssh minio-1 -c "curl -s
14    'http://192.168.56.10:3100/loki/api/v1/labels'"
15
16  vagrant ssh minio-1 -c "curl -s
17    'http://192.168.56.10:3200/api/search/tags'"
18 }
```

```
17
18 run_storage_evidence() {
19     vagrant ssh minio-1 -c "curl -sS -X POST
20         http://192.168.56.10:3100/flush"
21     vagrant ssh minio-1 -c "curl -sS -X POST
22         http://192.168.56.10:9009/ingester/flush"
23     sleep 30
24
25     for bucket in loki-data tempo-data mimir-blocks; do
26         echo "--- local/$bucket ---"
27         vagrant ssh minio-1 -c "sudo -u vagrant mc ls --recursive \
28 local/$bucket --insecure | head -20"
29     done
30 }
```

Listado 6.18: Extracto de `test-provision.sh`: comprobaciones de ingesta y de persistencia en MinIO

Validación y pruebas

ESTE capítulo recoge las pruebas realizadas para verificar que el sistema descrito en los capítulos 5 y 6 cumple los requisitos planteados: que el *pipeline* de observabilidad funciona de extremo a extremo con datos reales, que el conjunto es reproducible sobre los tres sistemas operativos soportados, y que el comportamiento del clúster de MinIO ante la caída de un nodo se corresponde con lo previsto en el diseño.

7.1 Validación *end-to-end* del *pipeline* de observabilidad

La validación de que las métricas, logs y trazas llegan realmente a su destino se ha realizado en dos niveles. Un primer nivel, más inmediato, consiste en consultar directamente las API HTTP de Mimir y Loki desde `minio-1`, mediante las dos consultas del listado 7.1: la primera pregunta a Mimir por las series que tiene en memoria y la segunda a Loki por las etiquetas que ha ingerido.

```
1 curl -s "http://192.168.56.10:9009/prometheus/api/v1/query?query=up"
2 curl -s "http://192.168.56.10:3100/loki/api/v1/labels"
```

Listado 7.1: Comprobación directa de ingesta en Mimir y Loki

Este nivel confirma que los *endpoints* responden y que existen series o etiquetas nuevas, pero no garantiza que los datos estén correctamente correlacionados entre sí. El segundo nivel, más exigente y el que se considera la validación real del objetivo 4 del TFG, consiste en consultar directamente Grafana: localizar una traza concreta generada por la aplicación TNS, y comprobar que su `traceID` permite recuperar, de forma correlacionada, tanto las métricas como los logs asociados a esa misma petición (el mecanismo de correlación traza-logs que lo hace posible se describe en la sección 6.2.6). La figura 7.1 muestra una de estas trazas en Grafana, junto con los logs recuperados a partir de su mismo `traceID`, y la figura 7.2, uno de los paneles de sistema aprovisionados automáticamente, alimentado desde Mimir.

La distinción entre ambos niveles no es teórica, ni el segundo es redundante respecto al primero. Durante las pruebas se dio el caso de que las tres señales se estaban ingiriendo correctamente (las API de Mimir y Loki respondían, Tempo almacenaba las trazas y los logs contenían el `traceID` correspondiente a cada una) y, pese a ello, la correlación no funcionaba en la interfaz de Grafana por un error en la construcción de la consulta LogQL asociada al *datasource* de Tempo (el diagnóstico completo se detalla en la sección 6.2.6). Es decir: el primer nivel de comprobación daba resultado satisfactorio mientras el objetivo 4 seguía sin cumplirse. De ahí que la validación se lleve hasta la interfaz de Grafana en lugar de detenerse en las API de ingesta.

Con el *datasource* ya corregido, esta comprobación de segundo nivel se ha repetido sobre los tres sistemas operativos soportados y para la aplicación TNS desplegada en ambos nodos `app-monitor-*`, con resultado satisfactorio en todos los casos (véase la sección 7.2).

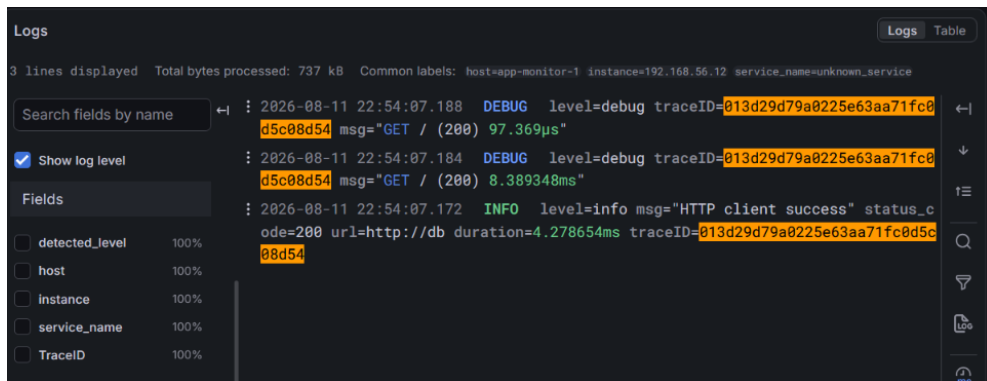


Figura 7.1: Correlación de una traza de TNS con sus logs en Grafana

7.1.1 Latencia de escritura en el *backend* S3

Durante las primeras pruebas se observó que, en algunos sistemas operativos, los objetos de Loki y Mimir no aparecían en los *buckets* de MinIO con la inmediatez de los de Tempo, lo que se interpretó como un posible fallo de ingesta. La causa real es más sencilla: Loki y Mimir mantienen los datos recientes en memoria y solo los vuelcan (*flush*) a MinIO de forma periódica (hasta 2 horas por defecto en ambos casos) [19, 20], mientras que Tempo lo hace con una cadencia mucho más corta. La ausencia de objetos nuevos no implica, por tanto, que Loki o Mimir no estén recibiendo datos. Para verificar la ingesta sin esperar al vaciado natural, ambos servicios exponen un extremo administrativo que lo fuerza, recogido en el listado 7.2.

```
1 curl -X POST http://192.168.56.10:3100/flush # Loki
2 curl -X POST http://192.168.56.10:9009/ingester/flush # Mimir
```

Listado 7.2: Forzado manual del vaciado a MinIO

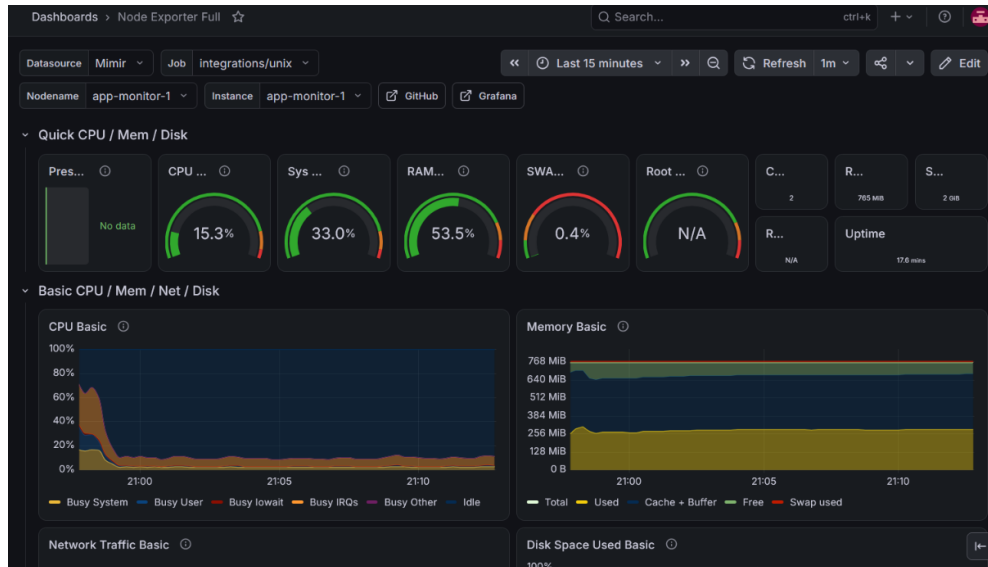


Figura 7.2: Panel *Node Exporter Full* en Grafana, con las métricas de sistema de *app-monitor-1* servidas desde Mimir

Conviene señalar que esto resuelve la verificación, pero no la automatiza: ambos extremos fuerzan el volcado en el momento en que se invocan, y hay que invocarlos a mano. La alternativa habría sido reducir en la configuración de Loki y de Mimir el intervalo de volcado por defecto, de dos horas, a un valor que hiciese visible la persistencia sin intervención; se descartó para no alterar el comportamiento por defecto de los servicios solo por comodidad de observación, pero es un ajuste razonable en un entorno de laboratorio como este.

7.2 Validación multi-sistema operativo

El conjunto completo del aprovisionamiento se ha ejecutado y validado de forma independiente sobre los tres sistemas operativos soportados: Debian, Rocky Linux y AlmaLinux. Cada validación cubre el ciclo entero, desde la selección del sistema operativo en `config.yml` hasta el despliegue de la aplicación TNS en ambos nodos `app-monitor-*`, y se apoya en el script `test-provision.sh`, explicado en la sección 6.4, que lo automatiza y deja un registro de cada ejecución.

La tabla 7.1 resume el resultado de cada comprobación sobre cada sistema operativo. Más relevante que el resultado, idéntico en las tres distribuciones, es en qué consiste cada comprobación y con qué evidencia se respalda, que es lo que recoge su segunda columna: cada fila indica el criterio concreto por el que esa comprobación se da por superada, y no solo que lo esté. Las cinco primeras filas son automáticas y las produce el propio script, con el detalle recogido en el listado 6.18: el aprovisionamiento completo se da por bueno si `site.yml`

termina sin tareas fallidas en ninguna de las cuatro máquinas; el estado del clúster de MinIO se lee de `mc admin info`; y la ingesta de cada señal se comprueba interrogando a Mimir, Loki y Tempo por lo que tienen almacenado, y listando después el contenido de sus *buckets* en MinIO para confirmar que los datos no se quedan en memoria. Las tres últimas filas, en cambio, exigen interacción con la interfaz de Grafana y no pueden derivarse de un registro de texto: se verifican manualmente y su evidencia son las capturas recogidas en este capítulo.

Comprobación	Evidencia y criterio de aceptación	Debian	Rocky	Alma
Aprovisionamiento completo (<code>site.yml</code>)	Registro del script: <code>site.yml</code> termina sin tareas fallidas en las cuatro máquinas	✓	✓	✓
Clúster MinIO distribuido operativo	<code>mc admin info</code> : los dos nodos y sus discos figuran en estado <i>online</i>	✓	✓	✓
Ingesta de métricas en Mimir	Consulta <code>query=up</code> con series activas y etiqueta <code>job</code> con entradas <code>tns/*</code> ; objetos en el <i>bucket</i> <code>mimir-blocks</code>	✓	✓	✓
Ingesta de logs en Loki	<code>/loki/api/v1/labels</code> devuelve las etiquetas de TNS; objetos en el <i>bucket</i> <code>loki-data</code>	✓	✓	✓
Ingesta de trazas en Tempo	<code>/api/search/tags</code> devuelve etiquetas de traza; objetos en el <i>bucket</i> <code>tempo-data</code>	✓	✓	✓
Correlación traza → logs en Grafana	Figura 7.1: desde una traza de TNS en Tempo se recuperan sus logs por el mismo <code>traceID</code>	✓	✓	✓
Correlación logs → traza en Grafana	Comprobación manual: el <code>traceID</code> extraído de una línea de log en Loki abre la traza correspondiente en Tempo	✓	✓	✓
Grafo de servicios (<i>metrics-generator</i>)	Comprobación manual: el grafo de Grafana muestra los servicios de TNS y las llamadas entre ellos	✓	✓	✓

Tabla 7.1: Matriz de validación por sistema operativo, con la evidencia que respalda cada comprobación

7.3 Comportamiento del clúster MinIO ante la caída de un nodo

Para verificar experimentalmente el comportamiento de **quórum** descrito en la sección 5.5, se ha detenido deliberadamente uno de los dos nodos del clúster de MinIO (`minio-2`) mientras el otro permanecía activo, y se ha comprobado el comportamiento resultante desde el cliente `mc`, desde el panel de MinIO en Grafana y desde los registros de los servicios que utilizan el clúster como *backend*.

La elección del nodo no es indiferente. Desde el punto de vista de MinIO ambos son intercambiables: el clúster carece de nodo maestro y cada uno aporta un único disco al mismo conjunto de codificación, de modo que el recuento de **quórum** resultante es idéntico caiga el

que caiga. Sin embargo, minio-1 aloja además la pila LGTM y es el extremo S3 al que apuntan Loki, Tempo y Mimir, así como el objetivo del que Grafana Alloy recolecta las métricas del clúster. Detener minio-2 deja intactos los medios de observación y limita el efecto al sujeto del experimento, de modo que lo que se mide es el comportamiento del almacenamiento ante la pérdida de un nodo y no una indisponibilidad del instrumental que lo registra:

- Las operaciones de **lectura** (listar *buckets*, listar objetos dentro de un *bucket*, descargar un objeto ya existente) continúan funcionando con normalidad con un único nodo activo.
- Las operaciones de **escritura**, tanto las subidas manuales como las escrituras automáticas de Loki, Tempo y Mimir hacia sus respectivos *buckets*, fallan con error hasta que el segundo nodo vuelve a estar disponible.
- El acceso a la **consola web** de MinIO se comporta de forma asimétrica: una sesión iniciada antes de la caída sigue siendo válida y permite seguir navegando por los *buckets*, pero un inicio de sesión nuevo falla y se presenta al usuario como un error de credenciales.

El propio clúster refleja la situación con claridad. El listado 7.3 recoge la salida de `mc admin info` con minio-2 detenido: el nodo aparece explícitamente como `offline`, la conectividad entre nodos baja a 1/2, y el resumen final indica un nodo fuera de servicio y un solo disco en línea de los dos que forman el conjunto de codificación por borrado.

```
1 $ sudo mc admin info local --insecure
2 o minio-1.local:9000
3   Uptime: 23 minutes
4   Version: 2025-09-07T16:13:09Z
5   Network: 1/2 OK
6   Drives: 1/1 OK
7   Pool: 1
8
9 o minio-2.local:9000
10  Uptime: offline
11  Drives: 0/1 OK
12
13 298 KiB Used, 5 Buckets, 16 Objects
14 1 node offline, 1 drive online, 1 drive offline, EC:1
```

Listado 7.3: Salida de `mc admin info` con minio-2 detenido

Esa misma información llega, sin intervención manual, a la plataforma de observabilidad desplegada en este TFG: las métricas nativas del clúster que Grafana Alloy recolecta de MinIO

y envía a Mimir alimentan el panel de MinIO aprovisionado automáticamente en Grafana. Es, en cierto modo, la comprobación cruzada más significativa del trabajo: la infraestructura de observabilidad detectando el fallo de la infraestructura de almacenamiento sobre la que ella misma se apoya.

El panel aporta además algo que la salida de `mc admin info` no contiene. Como muestra la figura 7.3, junto al número de discos en línea del conjunto de codificación por borrado se publican los dos umbrales de **quórum** del propio conjunto, y ahí la aritmética queda a la vista: con un solo disco activo, el clúster iguala el quórum de lectura (`Read Quorum = 1`) pero queda por debajo del de escritura (`Write Quorum = 2`). El indicador de estado del conjunto acompaña esa lectura pasando a cero. Es decir, la asimetría entre lecturas y escrituras con la que se abre esta sección no se deduce del comportamiento observado, sino que el propio clúster la publica como métrica.

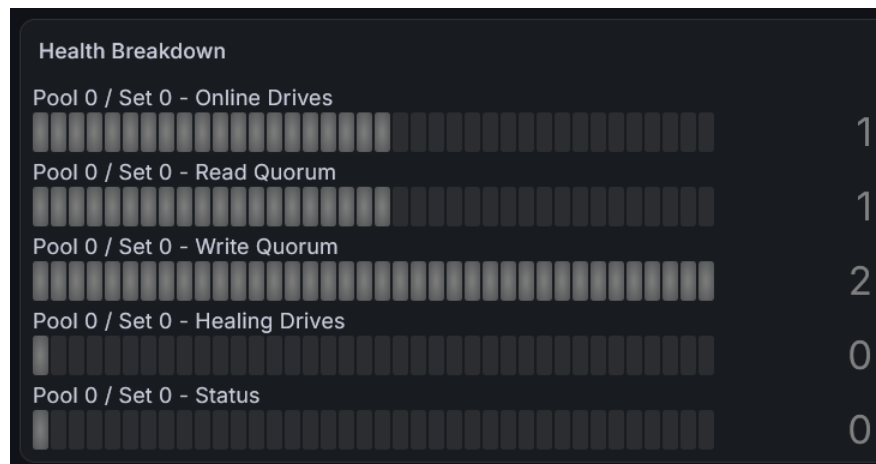


Figura 7.3: Panel Health Breakdown del *dashboard* de MinIO en Grafana con `minio-2` detenido: un disco en línea, igualando el quórum de lectura y por debajo del de escritura

Ahora bien, todo lo anterior describe el estado del clúster y las condiciones que este debe cumplir, no lo que les ocurre a sus usuarios: en ningún punto se indica que alguna escritura concreta haya sido rechazada, ni qué error recibe quien la intenta. El efecto funcional se observa desde los clientes del almacenamiento. Forzando el vaciado de Loki y Mimir con los extremos administrativos descritos en la sección 7.1, para no esperar a su cadencia natural, los tres servicios registran el fallo al intentar escribir en sus *buckets*. El listado 7.4 recoge esos registros: son la salida de `docker logs` para cada uno de los tres contenedores del stack, consultada en `minio-1` justo después de forzar el volcado.

```

1 # docker logs loki
2 level=error caller=flush.go:262 component=ingester msg="failed to
   flush"
3 err="failed to flush chunks: store put chunk: operation error S3:

```

```

4   PutObject, exceeded maximum number of attempts, 3, https response
   error
5   StatusCode: 503, api error SlowDownWrite: Resource requested is
6   unwritable, please reduce your request rate, num_chunks: 1"
7
8 # docker logs mimir
9 caller=shipper.go:167 level=error msg="uploading new block to long-term
10 storage failed" err="failed to upload index: upload s3 object:
   Resource
11 requested is unwritable, please reduce your request rate"
12
13 # docker logs tempo
14 level=error caller=complete_block_lifecycle.go:228 msg="failed to flush
15 complete block" err="error copying block from local to remote
   backend:
16 error writing object to s3 backend, object data.parquet: Resource
17 requested is unwritable, please reduce your request rate"

```

Listado 7.4: Errores de escritura hacia MinIO registrados por Loki, Mimir y Tempo con minio-2 detenido

El mensaje que MinIO devuelve en los tres casos es el mismo, y merece comentario porque resulta engañoso: `SlowDownWrite: Resource requested is unwritable, please reduce your request rate` sugiere una limitación por exceso de peticiones e invita a espaciar las escrituras, cuando la causa real es la pérdida del quórum de escritura por la ausencia de un nodo. Reducir el ritmo de escritura, que es lo que el texto propone, no habría corregido nada. El diagnóstico correcto solo aparece al cruzar ambas fuentes: el estado que reporta el clúster explica el error que observan sus clientes, y ninguna de las dos, por separado, lo dice todo.

El comportamiento de la consola web de MinIO merece el mismo comentario, porque repite el patrón sobre una superficie distinta. Con un nodo caído, las credenciales de administración siguen siendo exactamente las mismas y la lectura desde mc funciona sin ninguna incidencia, pero la interfaz rechaza el inicio de sesión como si fueran incorrectas; una sesión abierta con anterioridad, en cambio, conserva el acceso y permite seguir consultando el contenido de los *buckets*. La evidencia apunta, por tanto, a que lo que la pérdida de quórum impide no es validar las credenciales, sino completar el establecimiento de una sesión nueva. El efecto sobre el diagnóstico vuelve a ser el mismo que con `SlowDownWrite`: quien intente entrar en la consola justo después del incidente concluirá que el problema está en sus credenciales y buscará en la dirección equivocada. Este comportamiento se ha observado de forma idéntica en Rocky Linux y en AlmaLinux, lo que indica que se trata de una propiedad del propio MinIO y no de la distribución subyacente.

Este resultado confirma, de forma experimental, el comportamiento de **quórum** esperado para un clúster de dos nodos: al exigir MinIO más de la mitad de los nodos disponibles para admitir escrituras (aritmética detallada en la sección 5.5 y la tabla 5.1), la pérdida de uno de los dos nodos deja al clúster en el límite del quórum mínimo de lectura, pero por debajo del quórum de escritura. Los umbrales que el propio clúster publica en la figura 7.3 coinciden con los previstos en esa tabla, de modo que la predicción teórica y la medición sobre el sistema desplegado se corresponden. La limitación es arquitectónica, inherente al tamaño del clúster, y no un fallo de configuración corregible dentro del alcance de este TFG.

7.4 Idempotencia del despliegue

Se ha comprobado también que una segunda ejecución consecutiva del comando que despliega todo el sistema (`ansible-playbook site.yml`), sin cambios en la configuración, no modifica el sistema ya desplegado. En el caso del stack LGTM, el despliegue mediante Docker Compose es idempotente por sí mismo: `docker compose up` compara la definición de cada servicio con el contenedor en ejecución y solo recrea los que han cambiado. Dado que el contenido de los ficheros de configuración montados desde el host no forma parte de esa comparación, la definición de cada servicio incorpora una etiqueta (*label*) con el *hash* de su configuración, de modo que un cambio real de configuración fuerza la recreación únicamente del servicio afectado, y una ejecución sin cambios no modifica ningún contenedor. El rol `grafana_alloy` aplica esta misma estrategia: en lugar de eliminar y recrear el contenedor en cada ejecución, lo declara con el módulo `docker_container` de Ansible y le añade una etiqueta (*label*) `config-hash` con el *hash* del fichero `config.alloy` montado, de modo que el contenedor solo se recrea cuando su configuración cambia realmente, sin sacrificar la **idempotencia** de las ejecuciones sin cambios. Así, una segunda ejecución consecutiva no recrea tampoco el contenedor de Alloy.

Medir esa idempotencia en lugar de darla por supuesta resultó, de hecho, imprescindible: la primera medición real reveló un punto de no idempotencia que ninguno de los dos mecanismos anteriores cubría, y que además se propagaba en cascada entre roles. El rol `minio` generaba el *token* de autenticación Prometheus del clúster con `mc admin prometheus generate` en cada ejecución, y lo reescribía en disco. Ese *token* es uno de los valores que el rol `grafana_alloy` inserta en la plantilla con la que genera `config.alloy`, de modo que un *token* nuevo producía un fichero de configuración distinto, un *hash* distinto y, por tanto, la recreación del contenedor de Alloy en cada ejecución. Es decir, una tarea marcada como no idempotente en un rol acababa reiniciando un contenedor gestionado por otro, tres roles más adelante. La corrección consiste en comprobar previamente si el fichero del *token* ya existe y omitir tanto su generación como su escritura en ese caso, de modo que el *token* se genera una

única vez, en el primer despliegue.

Una segunda medición, ya sobre Rocky Linux 9, sacó a la luz un caso distinto y más sutil, esta vez en el rol `firewall`. Su maniobra de neutralización de `firewalld` (arrancar el servicio para que reclame su tabla `nftables` persistente y pararlo a continuación, descrita en la sección 6.3) se ejecutaba de forma incondicional, de modo que cada ejecución volvía a arrancar un servicio que la anterior había dejado parado, y a pararlo de nuevo. Eso suponía dos tareas marcadas como `changed` por máquina y por `play` y, dado que el rol aparece en tres `plays` de `site.yml`, una segunda ejecución no podía dar `changed=0` en ninguna máquina de la familia RedHat.

A diferencia del caso del `token`, aquí el problema no es una tarea que reescribe un fichero con contenido equivalente, sino una cuya acción consiste precisamente en modificar el estado del sistema. Por eso la solución no puede ser silenciar el aviso: marcar las dos tareas con `changed_when: false` habría bastado para que Ansible dejase de contabilizarlas como cambios en su resumen final, pero el estado del sistema seguiría cambiando en cada ejecución y el informe estaría describiendo una idempotencia declarada, no real. La corrección consiste en sondear primero el estado de partida (si el servicio está activo, si arrancaría en el próximo inicio, y si queda una tabla `nftables` suya huérfana en el kernel) y condicionar la maniobra a que se dé alguna de esas tres situaciones. Cuando el cortafuegos ya está neutralizado, las dos tareas se omiten; cuando no lo está, se ejecutan y se reportan como cambio, que es justo la información que interesa conservar.

Ambos casos apuntan a la misma conclusión metodológica: la **idempotencia** de un despliegue no se deduce leyendo los roles, se mide ejecutándolos dos veces seguidas y observando el resultado. El listado 7.5 recoge el resumen de una segunda ejecución consecutiva sin cambios, ya con las dos correcciones aplicadas.

```

1 TASK [firewall : Informar del estado de firewalld]
   *****
2 ok: [minio-1] =>
3   msg: 'firewalld ya esta neutralizado (parado, deshabilitado y sin
4         tabla nftables residual): no se toca nada'
5
6 TASK [firewall : Arrancar firewalld (para que reclame su tabla ...)]
   ***
7 skipping: [minio-1]
8
9 TASK [firewall : Detener y deshabilitar firewalld (Rocky/Alma)]
   *****
10 skipping: [minio-1]
11 [...]
12
```

```

13 TASK [minio : Comprobar si ya existe el token de Prometheus]
    *****
14 ok: [minio-1]
15
16 TASK [minio : Generar token Prometheus del cluster MinIO]
    *****
17 skipping: [minio-1]
18
19 TASK [minio : Guardar bearer_token del cluster MinIO]
    *****
20 skipping: [minio-1]
21 [...]
22
23 TASK [grafana_alloy : Crear configuracion de Alloy]
    *****
24 ok: [app-monitor-1]
25 ok: [app-monitor-2]
26
27 TASK [grafana_alloy : Ejecutar contenedor Alloy]
    *****
28 ok: [app-monitor-2]
29 ok: [app-monitor-1]
30 [...]
31
32 TASK [lgtm_stack : Levantar stack LGTM con docker compose]
    *****
33 ok: [minio-1]
34
35 PLAY RECAP *****
36 app-monitor-1 : ok=37  changed=0  unreachable=0  failed=0  skipped=6
37 app-monitor-2 : ok=34  changed=0  unreachable=0  failed=0  skipped=8
38 minio-1       : ok=88  changed=0  unreachable=0  failed=0  skipped=15
39 minio-2       : ok=49  changed=0  unreachable=0  failed=0  skipped=9

```

Listado 7.5: Extracto y resumen final de una segunda ejecución de `site.yml` sin cambios, sobre Rocky Linux 9.6

Ninguna de las cuatro máquinas registra cambios (`changed=0`), lo que confirma que el despliegue ha convergido: el estado actual del sistema ya coincide con el estado deseado y declarado en los roles de Ansible, y una ejecución adicional no lo altera. El extracto incluye además las tareas implicadas en los dos problemas descritos más arriba. Del rol `firewall`, el sondeo que constata que el cortafuegos ya está neutralizado y las dos tareas de la maniobra,

ahora omitidas. Del rol `minio`, la comprobación previa del fichero del *token* junto a su generación y escritura, también omitidas, y las dos tareas de Alloy que antes se veían arrastradas por ellas y que ya no recrean el contenedor.

Conclusiones

ESTE último capítulo cierra la memoria recapitulando el trabajo realizado. Se revisa en primer lugar el grado de cumplimiento de los objetivos planteados en el capítulo 1; se recogen después las lecciones aprendidas y los conocimientos adquiridos durante el desarrollo, junto con su relación con las competencias de la mención cursada; y se apuntan, por último, las líneas de trabajo futuro que quedan abiertas.

8.1 Situación final del trabajo

Los cuatro objetivos planteados en la introducción de este TFG se han alcanzado. Se ha diseñado y justificado una arquitectura lógica de cuatro nodos (capítulo 5); se ha automatizado por completo su aprovisionamiento mediante Vagrant y Ansible, de forma reproducible sobre Debian, Rocky Linux y AlmaLinux (capítulo 6); se ha desplegado un clúster distribuido de MinIO junto a un stack de observabilidad LGTM completo, reescrito desde una primera implementación mediante *playbooks* planos hacia una organización en roles de Ansible reutilizables (también capítulo 6); y se ha validado el conjunto mediante un caso de uso práctico (la aplicación de demostración TNS, desplegada en ambos nodos de aplicación), comprobando de forma experimental que la telemetría generada se recolecta, almacena y correlaciona correctamente de extremo a extremo (capítulo 7).

8.2 Lecciones aprendidas

El proceso de garantizar la compatibilidad del proyecto con las tres distribuciones soportadas (Debian, Rocky Linux y AlmaLinux) ha sido, en términos de esfuerzo real invertido, la parte más costosa del trabajo, y también la que ha dejado las lecciones más transferibles. Ninguno de los problemas encontrados (la ubicación real del binario de MinIO, el comportamiento de las tablas persistentes de `nftables` de `firewalld`, la ruptura de compatibilidad

entre versiones de `requests` y la colección `community.docker`, o las diferencias en el `secure_path` de `sudo`) estaba documentado de forma directa y accesible; todos requirieron un proceso de diagnóstico basado en evidencia (registros del sistema, herramientas de diagnóstico de red de bajo nivel como `nft list ruleset`, comparación de rutas de instalación entre distribuciones) más que en la consulta directa de documentación oficial. La portabilidad entre plataformas no se obtiene, por tanto, por el mero hecho de usar herramientas de IaC: hay que verificarla y, en muchos casos, corregirla plataforma por plataforma.

La refactorización hacia roles de Ansible ha dejado, además, una lección de mantenibilidad concreta. Empezar por una implementación en *playbooks* planos, y solo reorganizarla en roles una vez comprobado que funcionaba, hizo tangible el problema que los roles resuelven: lógica idéntica repetida entre *playbooks*, como la secuencia de neutralización de `firewalld` que aparecía copiada en los tres que la necesitaban y que en la versión de roles pasa a definirse una sola vez y reutilizarse como dependencia. Haber medido ese coste sobre una primera versión real, en lugar de asumirlo en abstracto, confirmó que el paso a roles no era un cambio cosmético sino una mejora efectiva de mantenibilidad, y es lo que justificó retirar por completo los *playbooks* planos una vez terminada la migración, dejando la versión basada en roles como única implementación vigente.

8.3 Conocimientos adquiridos

Más allá de los resultados del proyecto, el desarrollo de este TFG ha supuesto un aprendizaje personal considerable, en varias áreas que apenas se tratan en las asignaturas del grado con este nivel de profundidad:

- **Automatización de infraestructura.** Uso en un proyecto real de Vagrant y, sobre todo, de Ansible: inventarios, módulos, plantillas Jinja2 y la organización del código en roles reutilizables, junto con la idempotencia como criterio de calidad de la automatización y no solo como propiedad teórica.
- **Contenedores.** Despliegue y operación de servicios con Docker y Docker Compose, incluidas cuestiones menos visibles que el uso básico: los modos de red (*bridge* frente a *host*) y sus consecuencias sobre la conectividad, la orquestación declarativa con Compose y su modelo de detección de cambios, o los límites que el aislamiento del contenedor impone a un servicio que necesita red y disco directos, como MinIO en modo distribuido.
- **Almacenamiento de objetos.** El modelo S3 como interfaz de facto de la industria, y el funcionamiento interno de un clúster distribuido de MinIO: codificación por borrado, *quórum* de lectura y de escritura, y sus consecuencias prácticas sobre la disponibilidad.

- **Observabilidad.** Los tres tipos de señal de telemetría y su correlación, la operación de un stack LGTM completo con sus lenguajes de consulta (PromQL, LogQL, TraceQL), y los modelos de recolección *push* y *pull* con un agente unificado como Grafana Alloy.
- **Administración de sistemas Linux multi-distribución.** Gestión de paquetes, servicios de systemd, redes y cortafuegos (firewalld, nftables) y particularidades como el `secure_path` de sudo, con las diferencias reales entre Debian y la familia RHEL como hilo conductor.

A esto se añade un aprendizaje transversal: la disciplina de documentar el sistema a medida que evoluciona, manteniendo una memoria técnica coherente con una infraestructura que cambia; y la de acudir a los canales reales de los proyectos de código abierto (*issues*, código fuente, notas de versión) cuando la documentación oficial no cubre el problema encontrado, como ocurrió con varios de los descritos en el capítulo 6.

8.4 Relación con las competencias de la mención

Este TFG se enmarca en la mención de Tecnologías de la Información, y pone en práctica de forma directa varias de sus competencias centrales: el diseño y la administración de infraestructuras distribuidas, la automatización de la configuración de sistemas mediante herramientas de IaC, y el despliegue y explotación de plataformas de observabilidad nativas de la nube sobre arquitecturas de contenedores. La necesidad de portar el sistema completo entre tres distribuciones Linux distintas ha requerido, además, un conocimiento operativo transversal (gestión de paquetes, redes, *firewall* y configuración de privilegios del sistema operativo) que va más allá del uso de una única herramienta o tecnología concreta.

8.5 Líneas futuras

El trabajo realizado deja abiertas varias líneas de continuación, algunas ya identificadas de forma explícita durante el desarrollo:

- **Alta disponibilidad de escritura en el clúster MinIO.** Como se discute en la sección 5.5, un clúster de dos nodos no puede tolerar la caída de un nodo sin perder disponibilidad de escritura, al tratarse de una limitación de *quórum* inherente a su tamaño. Ampliar el número de dominios de fallo permitiría validar un escenario de alta disponibilidad real: manteniendo el disco único por nodo de este despliegue harían falta cuatro nodos, aunque la misma tolerancia se consigue con tres nodos que aporten dos discos cada uno. Cualquiera de las dos opciones exige un consumo de recursos que excede las posibilidades del entorno de laboratorio actual, y resultaría especialmente interesante

llevando el proyecto a un entorno de producción sobre un clúster de servidores físicos reales, donde los dominios de fallo son máquinas independientes de verdad y no máquinas virtuales que comparten un mismo anfitrión.

- **Endurecimiento de la seguridad del despliegue.** La sección de alcance y seguridad del capítulo 5 documenta las simplificaciones asumidas deliberadamente en este laboratorio: credenciales por defecto en texto plano, ausencia de TLS entre componentes y desactivación completa de `firewalld`. La línea de continuación natural consiste en revertir cada una de ellas: gestionar los secretos con Ansible Vault, habilitar TLS en MinIO y en los componentes del stack LGTM, y sustituir la desactivación del cortafuegos por una apertura selectiva de los puertos que cada rol necesita.
- **Instrumentación OpenTelemetry de una aplicación propia.** La aplicación de demostración TNS ya está instrumentada para Jaeger y envía sus trazas directamente a Tempo, sin pasar por el receptor OTLP que Grafana Alloy expone en cada nodo (sección 5.3.1), de modo que ese camino del *pipeline* está desplegado pero sin ejercitar. Recurrir a una aplicación ya instrumentada fue una decisión de alcance: desarrollar una desde cero habría desplazado el peso del trabajo desde la infraestructura hacia el desarrollo de la propia aplicación. Retomar esa tarea, instrumentando una aplicación propia con el SDK de OpenTelemetry, permitiría validar el flujo OTLP completo (de la aplicación a Alloy, y de este a Tempo) y cubrir la parte de instrumentación desde cero que TNS da por hecha.
- **Alertas y objetivos de nivel de servicio.** La plataforma actual es de observación pasiva: las señales se recolectan y se consultan, pero nadie es avisado cuando algo va mal. Definir reglas de alerta sobre las métricas de Mimir (por ejemplo, sobre la tasa de errores o la latencia de la aplicación, o sobre el estado del clúster MinIO) y canales de notificación mediante Grafana Alerting convertiría el despliegue en un sistema de monitorización activa.
- **Integración continua del código de infraestructura.** La validación multi-sistema operativo descrita en el capítulo 7 se ejecuta hoy de forma manual mediante `scripts/test-provision.sh`. Incorporar análisis estático de los roles (`ansible-lint`) y pruebas automatizadas por rol mediante Ansible Molecule en un flujo de integración continua convertiría esas comprobaciones en pruebas de regresión automáticas, ejecutadas en cada cambio del repositorio.

Lista de acrónimos

CLI Command Line Interface. [10](#)

DHCP Dynamic Host Configuration Protocol. [34](#)

ECTS European Credit Transfer and Accumulation System. [24](#)

HA High Availability (alta disponibilidad). [35](#)

IaC Infrastructure as Code. [1](#), [2](#), [4](#), [8](#), [10](#), [11](#), [21](#), [29](#), [38](#), [79](#)

LGTM Loki, Grafana, Tempo, Mimir. [2](#), [4](#)

NAT Network Address Translation. [34](#)

OTLP OpenTelemetry Protocol. [18](#)

S3 Simple Storage Service. [1](#), [4](#), [15](#)

SDK Software Development Kit. [49](#), [61](#), [80](#)

TFG Trabajo de Fin de Grado. [1](#), [2](#)

TNS aplicación de demostración de Grafana Labs (The New Stack). [14](#), [18](#), [53](#)

TSDB Time Series Database (base de datos de series temporales). [18](#)

VM Virtual Machine (máquina virtual). [10](#)

Glosario

erasure coding Técnica de codificación por borrado que divide los datos en fragmentos y añade fragmentos de paridad, permitiendo reconstruir la información original aunque se pierda parte de los fragmentos, sin necesidad de replicar los datos completos. [4](#), [16](#), [36](#)

idempotencia Propiedad de una operación por la cual ejecutarla varias veces produce el mismo resultado que ejecutarla una sola vez, sin efectos secundarios adicionales en las ejecuciones repetidas. [42](#), [64](#), [73](#), [74](#)

observabilidad Capacidad de inferir el estado interno de un sistema a partir de las señales que este expone hacia el exterior, habitualmente métricas, logs y trazas. [1](#), [5](#), [6](#)

quórum Número mínimo de nodos que deben estar disponibles en un sistema distribuido para que este admita una operación (por ejemplo, una escritura) de forma segura. [15](#), [16](#), [35](#), [60](#), [69](#), [71](#), [73](#), [78](#), [79](#)

Bibliografía

- [1] C. Majors, L. Fong-Jones, and G. Miranda, *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, 2022.
- [2] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. O'Reilly Media, 2020.
- [3] Amazon Web Services, Inc., “Amazon simple storage service – api reference,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html>
- [4] MinIO, Inc., “Minio object storage documentation (código fuente),” 2026, desde el 10 de octubre de 2025 la documentación de MinIO Object Storage dejó de publicarse en la web y se distribuye como código fuente para que la comunidad la compile localmente, que es como se ha consultado aquí. consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://github.com/minio/docs>
- [5] —, “Aistor – minio object store for ai,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://www.min.io/product/aistor>
- [6] Ceph Foundation, “Ceph documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://docs.ceph.com/en/latest/>
- [7] Prometheus Authors, “Prometheus documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://prometheus.io/docs/introduction/overview/>
- [8] Grafana Labs, “Grafana lgtm stack – loki, grafana, tempo, mimir,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://grafana.com/docs/>
- [9] —, “Grafana alloy documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://grafana.com/docs/alloy/latest/>

- [10] OpenTelemetry Authors, “Opentelemetry documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/>
- [11] HashiCorp, “Vagrant documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://developer.hashicorp.com/vagrant/docs>
- [12] Red Hat, Inc., “Ansible documentation – roles,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html
- [13] Oracle Corporation, “Oracle virtualbox user manual,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://www.virtualbox.org/manual/>
- [14] Pallets Projects, “Jinja documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://jinja.palletsprojects.com/>
- [15] Docker, Inc., “Docker documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://docs.docker.com/>
- [16] —, “Docker compose documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://docs.docker.com/compose/>
- [17] MinIO, Inc., “Erasure coding – minio object storage documentation,” 2026, fuente reStructuredText de la documentación community, compilada localmente (véase [4]). consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://github.com/minio/docs/blob/main/source/operations/concepts/erasure-coding.rst>
- [18] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. [En línea]. Disponible en: <https://sre.google/sre-book/table-of-contents/>
- [19] Grafana Labs, “Grafana loki documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://grafana.com/docs/loki/latest/>
- [20] —, “Grafana mimir documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://grafana.com/docs/mimir/latest/>
- [21] —, “Grafana tempo documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://grafana.com/docs/tempo/latest/>
- [22] —, “The new stack (tns) observability demo application,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://github.com/grafana/tns>

- [23] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. [En línea]. Disponible en: <https://git-scm.com/book/>
- [24] YAML Language Development Team, “Yaml ain’t markup language (yaml) version 1.2.2,” 2021, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://yaml.org/spec/1.2.2/>
- [25] K. Schwaber and J. Sutherland, “The scrum guide,” 2020, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://scrumguides.org/scrum-guide.html>
- [26] firewallld project, “firewalld documentation,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://firewalld.org/documentation/>
- [27] Netfilter Project, “nftables wiki,” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://wiki.nftables.org/>
- [28] ansible-collections/community.docker contributors, “Not supported url scheme http+docker (issue #860),” 2026, consultado el 9 de septiembre de 2026. [En línea]. Disponible en: <https://github.com/ansible-collections/community.docker/issues/860>